PRACTICAL – 01
Environment Setup and JavaScript Basics

1(a): Install Node.js and Visual Studio Code, and verify installation.

Step 1: Install Node.js

1. Download and install Node.js from the official Node.js website.
2. Run the installer and follow the installation steps.
3. After installation, open Command Prompt.
4. Verify Node.js using: node -v
5. Verify npm using: npm -v

```
C:\mern>node -v
v22.14.0

C:\mern>npm -v
10.9.2
```

Step 2: Install Visual Studio Code

1. Download and install Visual Studio Code.
2. Open Visual Studio Code after installation.
3. Create or open a project folder.
4. Open the VS Code terminal using: Terminal → New Terminal
5. Verify Node.js and npm from the VS Code terminal:

```
PS C:\mern> node -v
v22.14.0
PS C:\mern> npm -v
10.9.2
```

1(b): Create a simple Node.js file and run it using the terminal.

Code:

```
console.log("Hello, Node.js!");
console.log("My first Node.js program");
```

Output:

```
C:\mern>node p1(b).js
Hello, Node.js!
My first Node.js program
```

1(c): Write a Node.js script demonstrating basic JavaScript commands (console log, variables, and arithmetic operations).

Code:

```
let a=15;
let b=25;
let sum=a+b;
```

```
console.log("First Number:",a);
console.log("Second Number:",b);
console.log("Sum:",sum);
```

Output:

```
C:\mern>node p1(c).js
First Number: 15
Second Number: 25
Sum: 40
```

1(d): Create and run a simple program using variables and functions.

Code:

```
function add(a,b){
  return a+b;
}
let res=add(10,20);
console.log("Sum= "+res);
```

Output:

```
C:\mern>node p1(d).js
Sum= 30
```

1(e): Demonstrate use of operators and control statements in JavaScript.

Code:

```
let marks=60;
if(marks>=40){
  console.log("Pass");
}
else{
  console.log("Fail");
}
```

Output:

```
C:\mern>node p1(e).js
Pass
```

1(f): Write programs using loops and template literals.

Code:

```
const prompt = require("prompt-sync")();
let n=Number(prompt("Enter a number:"));
console.log("Table of "+n);
for(let i=1; i<=10; i++)
{
  console.log(`${n}*${i}=${n*i}`);
}
```

Output:

```
C:\mern>node p1(f).js
```

```
Enter a number:10
Table of 10
10*1=10
10*2=20
10*3=30
10*4=40
10*5=50
10*6=60
10*7=70
10*8=80
10*9=90
10*10=100
```

PRACTICAL – 02
Core JavaScript Concepts for MERN

2(a): Create functions and call them with parameters.

```
const prompt = require("prompt-sync")();
let a = Number(prompt("Enter first number: "));
let b = Number(prompt("Enter second number: "));
function add(x, y) {
  return x + y;
}
let result = add(a, b);
console.log("Sum =", result);
```

Output:

```
D:\MERN\MERN Project>node 2a.js
Enter first number: 54
Enter second number: 76
Sum = 130
```

2(b): Work with arrays and objects in JavaScript.

```
const prompt = require("prompt-sync")();
let fruits = [];
for (let i = 0; i < 3; i++) {
  fruits.push(prompt("Enter fruit " + (i + 1) + ": "));
}
let student = {
  name: prompt("Enter student name: "),
  age: Number(prompt("Enter age: ")),
  course: prompt("Enter course: ")
};
console.log("\nFruits:");
console.log(fruits);
console.log("\nStudent Details");
console.log(student);
```

Output:

```
D:\MERN\MERN Project>node 2b.js
Enter fruit 1: Apple
Enter fruit 2: Orange
Enter fruit 3: Mango
Enter student name: Naresh
Enter age: 25
Enter course: BSCIT

Fruits:
[ 'Apple', 'Orange', 'Mango' ]

Student Details
{ name: 'Naresh', age: 25, course: 'BSCIT' }
```

2(c): Demonstrate use of arrow functions.

```
const prompt = require("prompt-sync")();
let num = Number(prompt("Enter a number: "));
const square = (n) => n * n;
console.log("Square =", square(num));
```

Output:

```
D:\MERN\MERN Project>node 2c.js
Enter a number: 40
Square = 1600
```

2(d): Write a program using array methods (map, filter, reduce).

```
const prompt = require("prompt-sync")();
let numbers = [];
for (let i = 0; i < 5; i++) {
  numbers.push(Number(prompt("Enter number " + (i + 1) + ": ")));
}
let doubled = numbers.map(num => num * 2);
let filtered = numbers.filter(num => num > 25);
let total = numbers.reduce((sum, num) => sum + num, 0);

console.log("Original Array:", numbers);
console.log("Doubled Array:", doubled);
console.log("Numbers greater than 25:", filtered);
console.log("Sum:", total);
```

Output:

```
D:\MERN\MERN Project>node 2d.js
Enter number 1: 23
Enter number 2: 65
Enter number 3: 45
Enter number 4: 23
Enter number 5: 12
Original Array: [ 23, 65, 45, 23, 12 ]
Doubled Array: [ 46, 130, 90, 46, 24 ]
Numbers greater than 25: [ 65, 45 ]
Sum: 168
```

2(e): Create a program that manipulates objects and displays output.

```
const prompt = require("prompt-sync")();
let employee = {
  id: Number(prompt("Enter Employee ID: ")),
  name: prompt("Enter Employee Name: "),
  salary: Number(prompt("Enter Salary: "))
};
employee.department = prompt("Enter Department: ");
employee.salary += 5000;
console.log("\nEmployee Details");
console.log(employee);
```

Output:

```
D:\MERN\MERN Project>node 2d.js
Enter number 1: 43
Enter number 2: 12
Enter number 3: 65
D:\MERN\MERN Project>node 2d.js
Enter number 1: 43
Enter number 2: 12
Enter number 3: 65
Enter number 4: 23
Enter number 5: 98
```

2(f): Implement a small JavaScript program combining functions, arrays, and objects.

```
const prompt = require("prompt-sync")();
let students = [];
let n = Number(prompt("Enter number of students: "));
for (let i = 0; i < n; i++) {
  let name = prompt("Enter student name: ");
  let marks = Number(prompt("Enter marks: "));
  students.push({
    name: name,
    marks: marks
  });
}
function display(list) {
  console.log("\nStudent Details");
  list.forEach(student => {
    console.log(student.name + " : " + student.marks);
  });
}
function average(list) {
  let total = list.reduce((sum, student) => sum + student.marks, 0);
  return total / list.length;
}
display(students);
console.log("Average Marks =", average(students).toFixed(2));
```

Output:

```
D:\MERN\MERN Project>node 2f.js
Enter number of students: 5
Enter student name: Aziz
Enter marks: 45
Enter student name: ALi
Enter marks: 65
Enter student name: Sarah
Enter marks: 55
Enter student name: Varun
```

```
Enter marks: 67
Enter student name: Tulsi
Enter marks: 68

Student Details
Aziz : 45
ALi : 65
Sarah : 55
Varun : 67
Tulsi : 68
Average Marks = 60.00
```

PRACTICAL – 03
React Introduction and JSX

3(a): Create a React application using Vite and display the message "Welcome to MERN Stack Development".

Software Requirements:

Node.js (LTS)
Visual Studio Code
React
Vite
npm

Step 1: Install Node.js

Step 2: Check the installation of Node.js and npm

Step 3: Open the terminal and create a React project using Vite. Follow the commands.

```
D:\>cd MERN

D:\MERN>cd "MERN Project"

D:\MERN\MERN Project>node -v
v24.17.0

D:\MERN\MERN Project>npm -v
11.13.0

D:\MERN\MERN Project>npx -v
11.13.0

D:\MERN\MERN Project>npm create vite@latest
```

```
D:\MERN>cd "MERN Project"

D:\MERN\MERN Project>npm create vite@latest

> npx
> create-vite

|
o  Project name:
|  New-Project-App
|
o  Package name:
|  new-project-app
|
o  Select a framework:
|  React
|
```

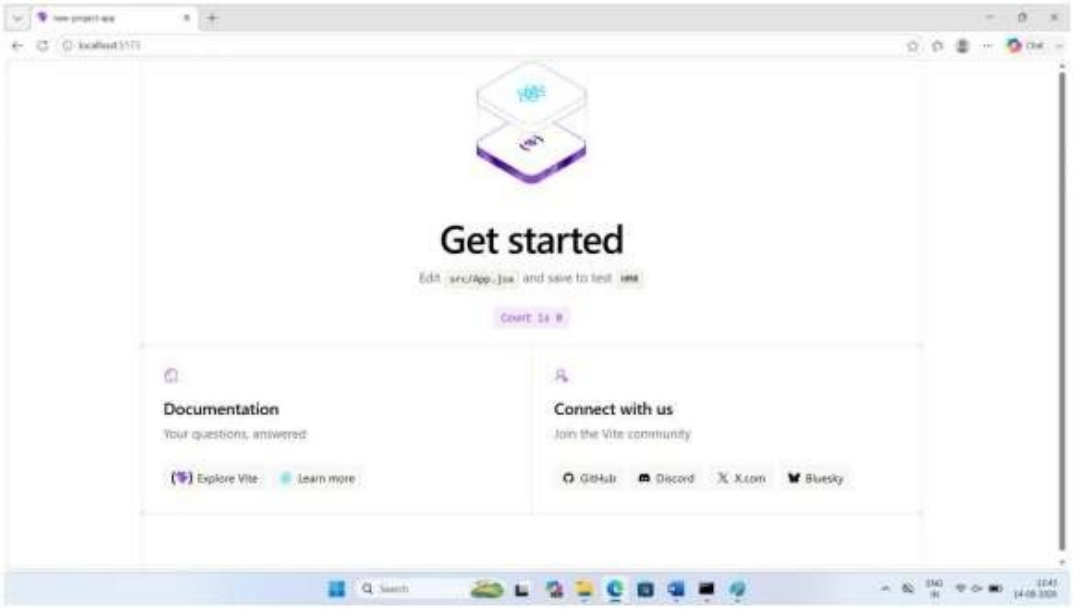
```
o  Select a variant:
|  JavaScript
|
o  Which linter to use?
|  ESLint
|
*  Install with npm and start now?
|  > Yes / No
```

```
VITE v8.2.1  ready in 1318 ms

➜  Local:   http://localhost:5173/
➜  Network: use --host to expose
➜  press h + enter to show help
```

Copy the URL : http://localhost:5173/

Open the browser and paste the URL.



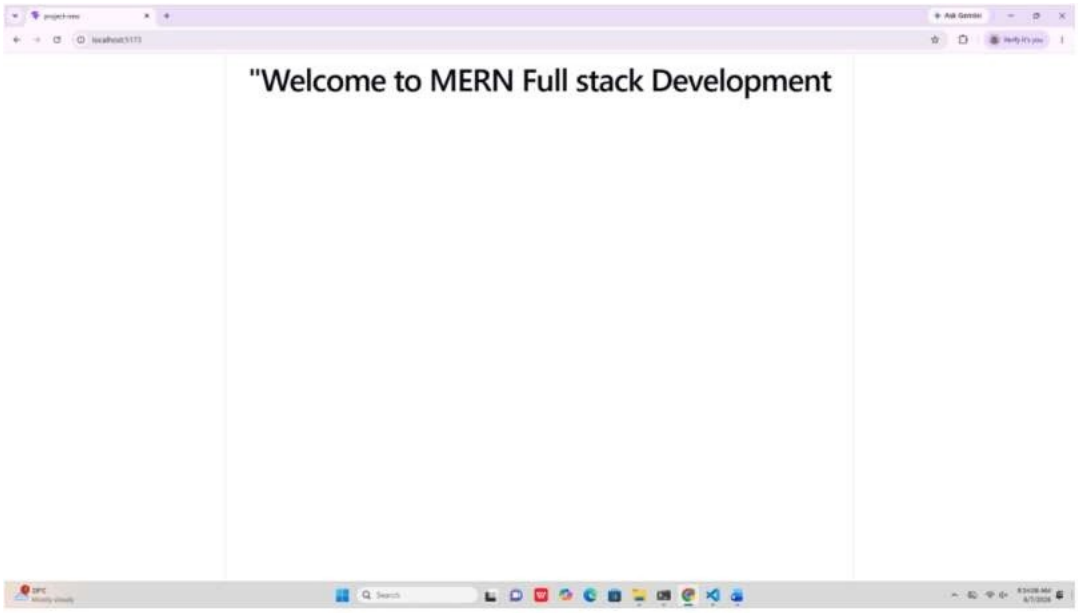
Step 4:

Open the App.js file and write the following code and save the content.

```
import "./App.css";

function App() {
  return (
    <div>
      <h1>"Welcome to MERN Full stack Development</h1>
    </div>
  );
}
export default App;
```

Output:



3(b): Display dynamic data using JSX.

```
import "./App.css";

function App() {
  const studentName = "Rahul Sharma";
  const course = "MERN Stack Development";
  const college = "ABC College";
  const age = 20;
  const marks = 89;

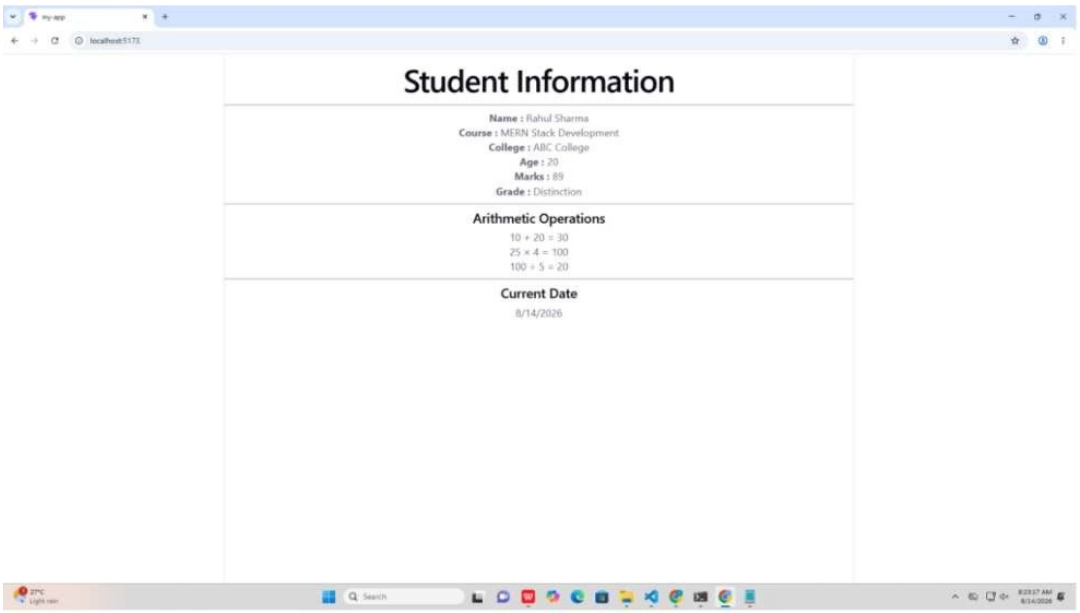
  function grade() {
    if (marks >= 75)
      return "Distinction";
    else if (marks >= 60)
      return "First Class";
    else
      return "Pass";
  }

  return (
    <div className="container">
      <h1>Student Information</h1>
      <hr />
      <p><b>Name :</b> {studentName}</p>
      <p><b>Course :</b> {course}</p>
      <p><b>College :</b> {college}</p>
      <p><b>Age :</b> {age}</p>
      <p><b>Marks :</b> {marks}</p>
      <p><b>Grade :</b> {grade()}</p>
      <hr />
      <h2>Arithmetic Operations</h2>
```

```
      <p>10 + 20 = {10 + 20}</p>
      <p>25 × 4 = {25 * 4}</p>
      <p>100 ÷ 5 = {100 / 5}</p>
      <hr />
      <h2>Current Date</h2>
      <p>{new Date().toLocaleDateString()}</p>
    </div>
  );
}

export default App;
```

Output:



3(c): Declare variables and write a function to calculate the sum of two numbers and display the result.

```
import { useState } from "react";
import "./App.css";

function App() {
  const [num1, setNum1] = useState("");
  const [num2, setNum2] = useState("");
  const [sum, setSum] = useState("");

  function calculateSum() {
    setSum(Number(num1) + Number(num2));
  }

  return (
    <div>
      <h1>Sum of Two Numbers</h1>

      <input
```

```
        type="number"
        placeholder="Enter First Number"
        value={num1}
        onChange={(e) => setNum1(e.target.value)}
      />
      <br /><br />

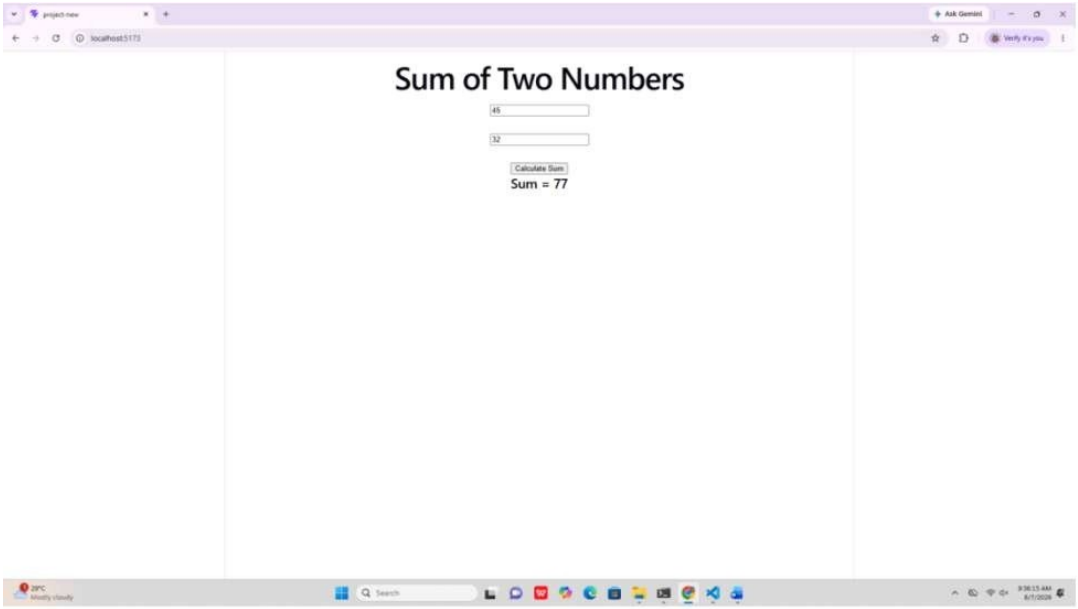
      <input
        type="number"
        placeholder="Enter Second Number"
        value={num2}
        onChange={(e) => setNum2(e.target.value)}
      />
      <br /><br />

      <button onClick={calculateSum}>Calculate Sum</button>

      <h2>Sum = {sum}</h2>
    </div>
  );
}

export default App;
```

Output:



3(d): Apply basic CSS styling to a React component.

App.jsx

```
import "./App.css";

function App() {
  return (
    <div className="container">
```

```
      <h1>Welcome to React</h1>
      <p>This is a styled React component.</p>
      <button>Click Me</button>
    </div>
  );
}

export default App;
```

App.css

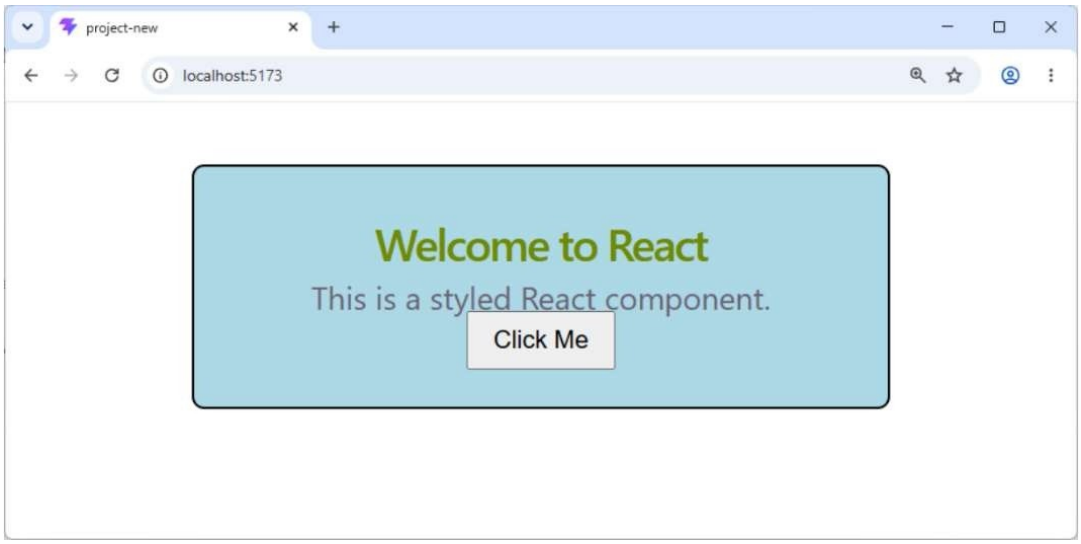
```
.container {
  width: 400px;
  margin: 50px auto;
  padding: 30px;
  text-align: center;
  border: 2px solid black;
  border-radius: 10px;
  background-color: lightblue;
}

h1 {
  color: darkblue;
}

p {
  font-size: 18px;
}

button {
  padding: 10px 20px;
  font-size: 16px;
}
```

Output:



3(e): Create a simple functional component and render it in App.js.

Hello.jsx

```
function Hello() {
  return (
    <div>
      <h2>Hello, React!</h2>
      <p>This is a simple functional component.</p>
    </div>
  );
}

export default Hello;
```

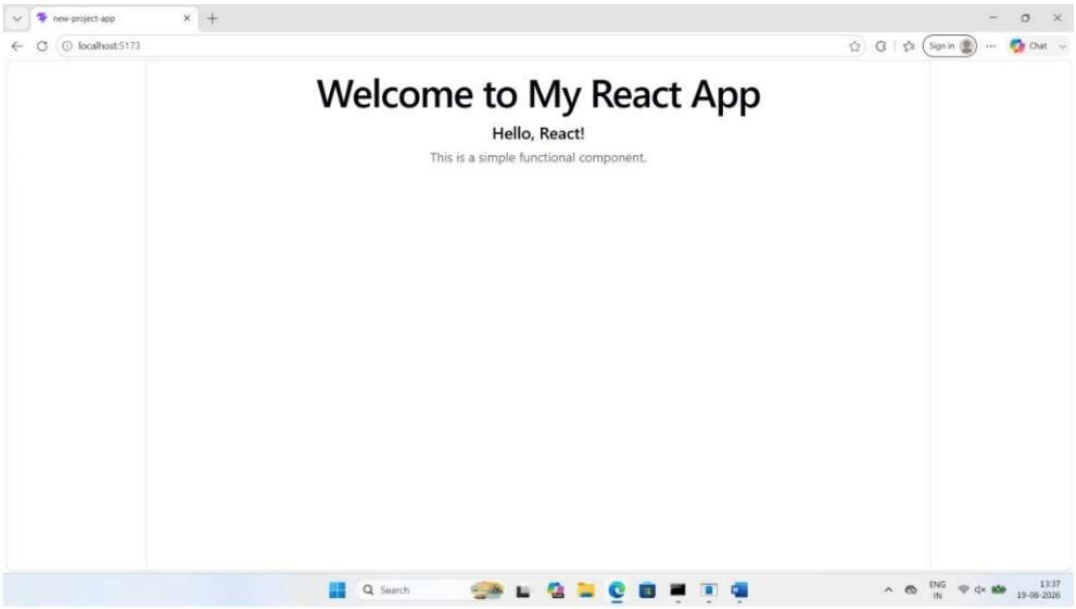
App.jsx

```
import Hello from "./Hello";

function App() {
  return (
    <div className="App">
      <h1>Welcome to My React App</h1>
      <Hello />
    </div>
  );
}

export default App;
```

Output:



3(f): Create a React component that displays the current date and time dynamically using JavaScript.

DateTime.jsx

```
import { useEffect, useState } from "react";

function DateTime() {
  const [dateTime, setDateTime] = useState(new Date());

  useEffect(() => {
    const timer = setInterval(() => {
      setDateTime(new Date());
    }, 1000);

    return () => clearInterval(timer);
  }, []);

  return (
    <div>
      <h2>Current Date and Time</h2>
      <p>{dateTime.toLocaleString()}</p>
    </div>
  );
}

export default DateTime;
```

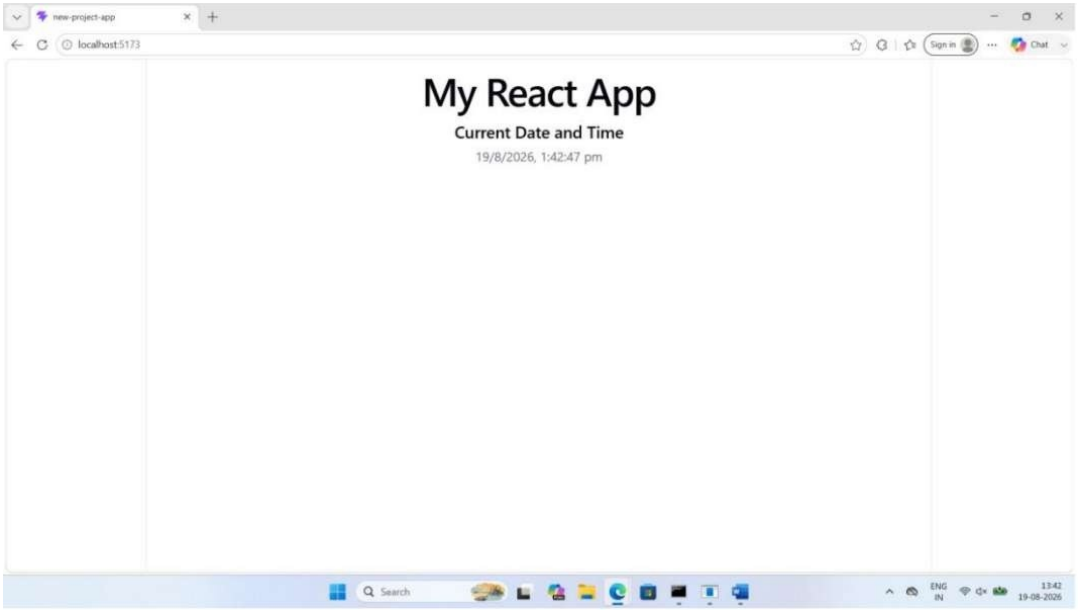
App.jsx

```
import DateTime from "./DateTime";

function App() {
  return (
    <div>
      <h1>My React App</h1>
      <DateTime />
    </div>
  );
}

export default App;
```

Output:


new-project-app
localhost:5173
Sign in
Chat
My React App
Current Date and Time
19/8/2026, 1:42:47 pm
Search
ENG
IN
13:42
19-08-2026

PRACTICAL – 04
React Components, Props, and State

4(a): Create multiple components and render them on a single page.

App.jsx

```
import React from "react";

function Header() {
  return <h1>My React App</h1>;
}

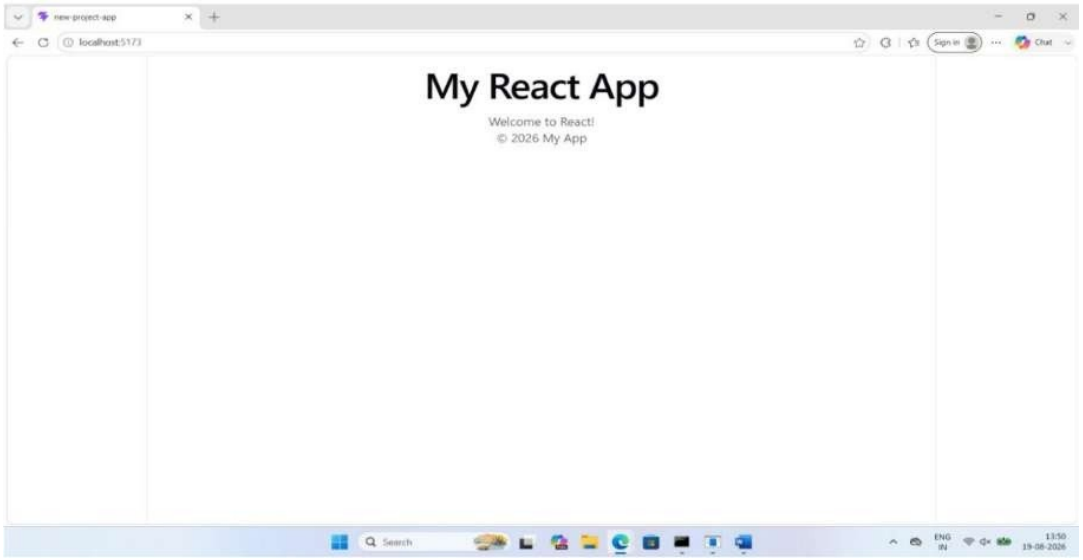
function Welcome() {
  return <p>Welcome to React!</p>;
}

function Footer() {
  return <p>© 2026 My App</p>;
}

function App() {
  return (
    <div>
      <Header />
      <Welcome />
      <Footer />
    </div>
  );
}

export default App;
```

Output:



4(b): Pass data between components using props.

```
import React from "react";

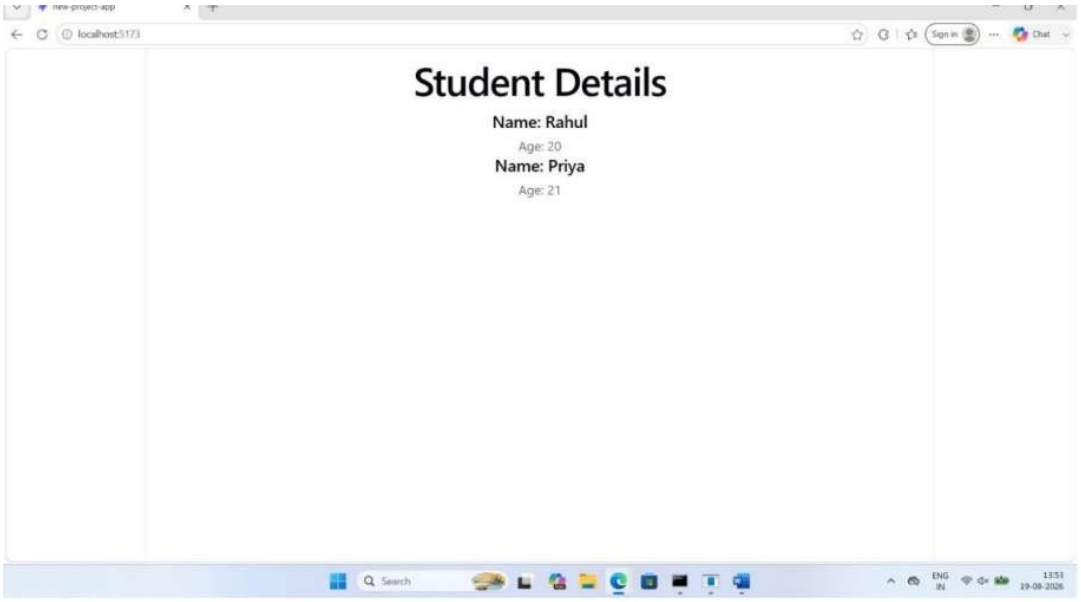
function Student({ name, age }) {
  return (
    <div>
      <h2>Name: {name}</h2>
      <p>Age: {age}</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <h1>Student Details</h1>

      <Student name="Rahul" age={20} />
      <Student name="Priya" age={21} />
    </div>
  );
}

export default App;
```

Output:



4(c): Use the useState hook to manage and update data.

```
import React, { useState } from "react";

function App() {
  const [name, setName] = useState("Guest");
```

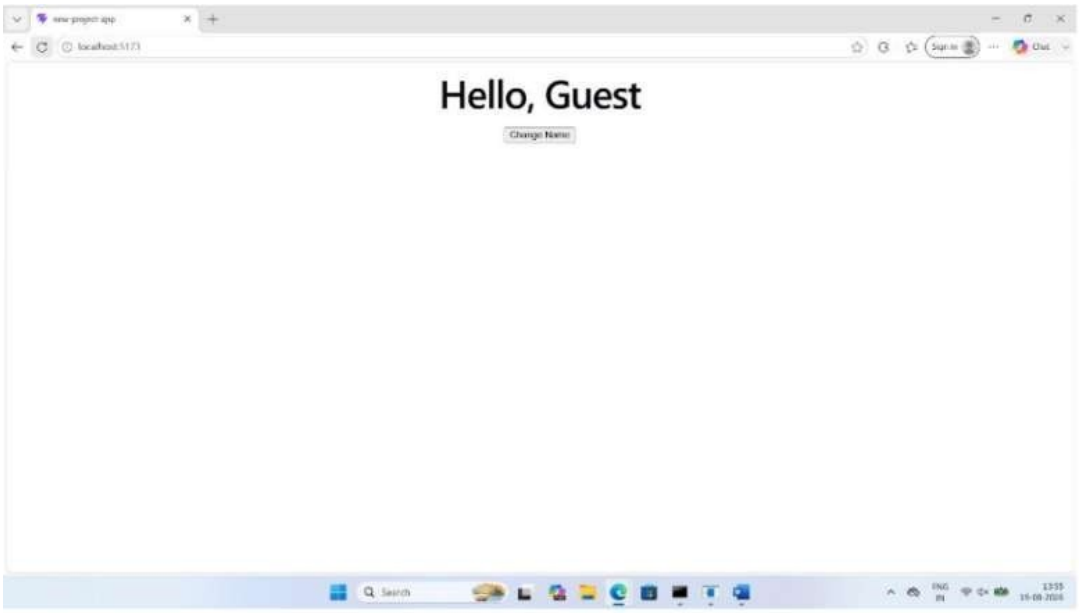
```
  const changeName = () => {
    setName("Student");
  };

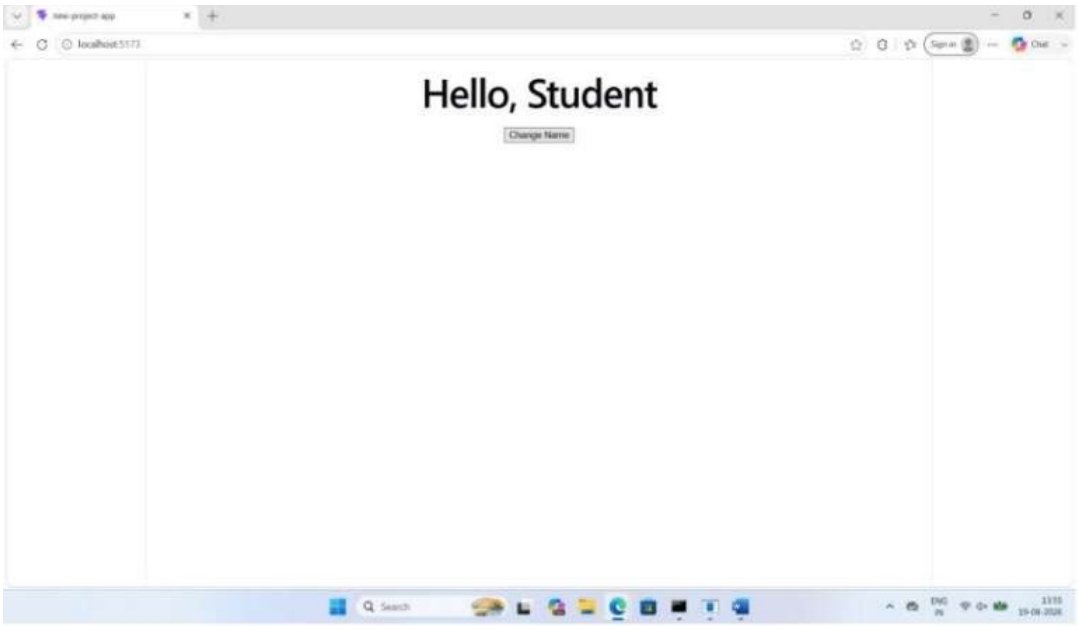
  return (
    <div>
      <h1>Hello, {name}</h1>

      <button onClick={changeName}>
        Change Name
      </button>
    </div>
  );
}

export default App;
```

Output:





4(d): Create a form component to accept user input and display it dynamically.

```
import React, { useState } from "react";

function UserForm() {
  const [name, setName] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
  };

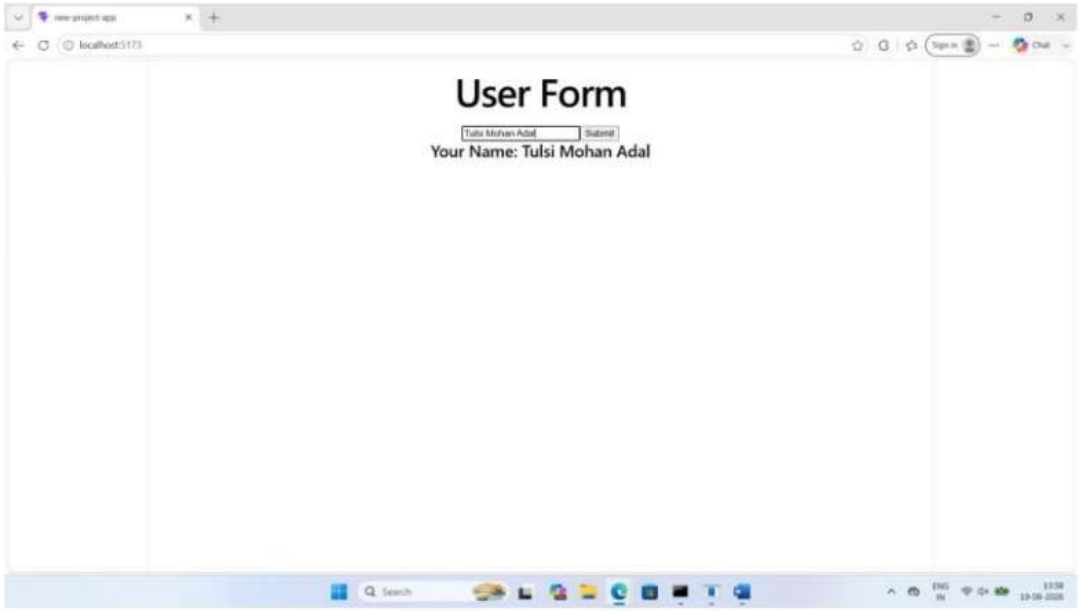
  return (
    <div>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Enter your name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
        <button type="submit">Submit</button>
      </form>
      <h2>Your Name: {name}</h2>
    </div>
  );
}

function App() {
  return (
    <div>
      <h1>User Form</h1>
      <UserForm />
    </div>
  );
```

```
}

export default App;
```

Output:



4(e): Display a list of items stored in the state using map().

```
import React, { useState } from "react";

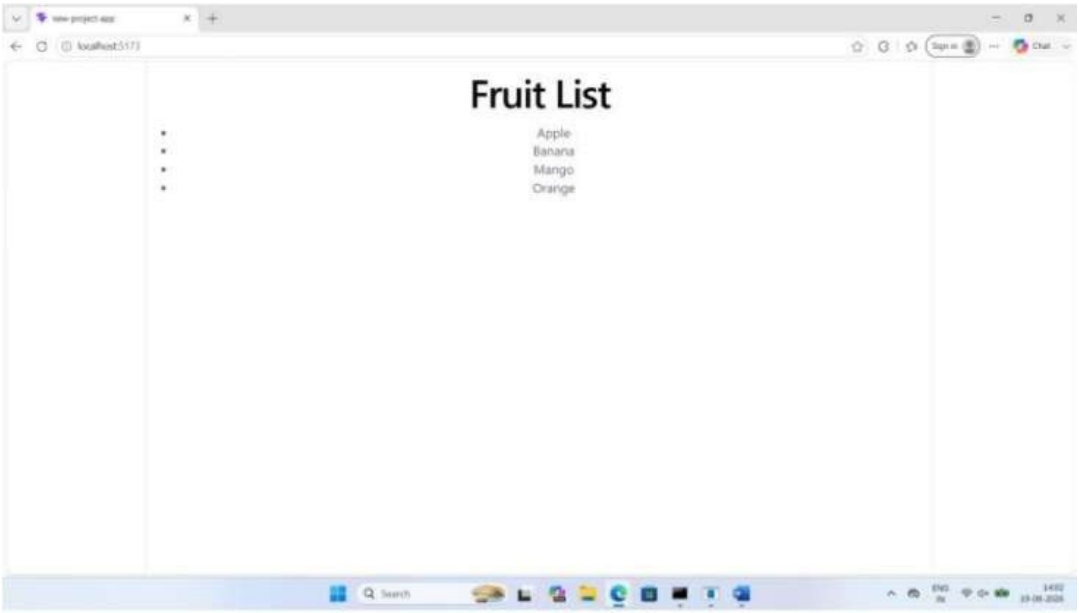
function App() {
  const [items] = useState([
    "Apple",
    "Banana",
    "Mango",
    "Orange"
  ]);

  return (
    <div>
      <h1>Fruit List</h1>

      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

Output:



4(f): Build a small interactive component (e.g., counter or to-do list).

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Counter: {count}</h1>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrease
      </button>

      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}

function App() {
  return (
    <div>
      <h1>Interactive Counter</h1>
      <Counter />
    </div>
```

```
  );
}

export default App;
```

Output:



PRACTICAL – 05
React Routing and Data Handling

5(a): Install and configure React Router in the React application.

Step 1: Install React Router

Open the terminal in your React project folder and run:

```
npm install react-router-dom
```

After installation, react-router-dom will be added to package.json.

Step 2: Configure BrowserRouter

Open: /src/main.jsx and replace the existing code with:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Step 3: Configure Routes

Open: /src/App.jsx

```
import { Routes, Route } from "react-router-dom";

function App() {
  return (
    <Routes>
      <Route path="/" element={<h1>Home Page</h1>} />
    </Routes>
  );
}

export default App;
```

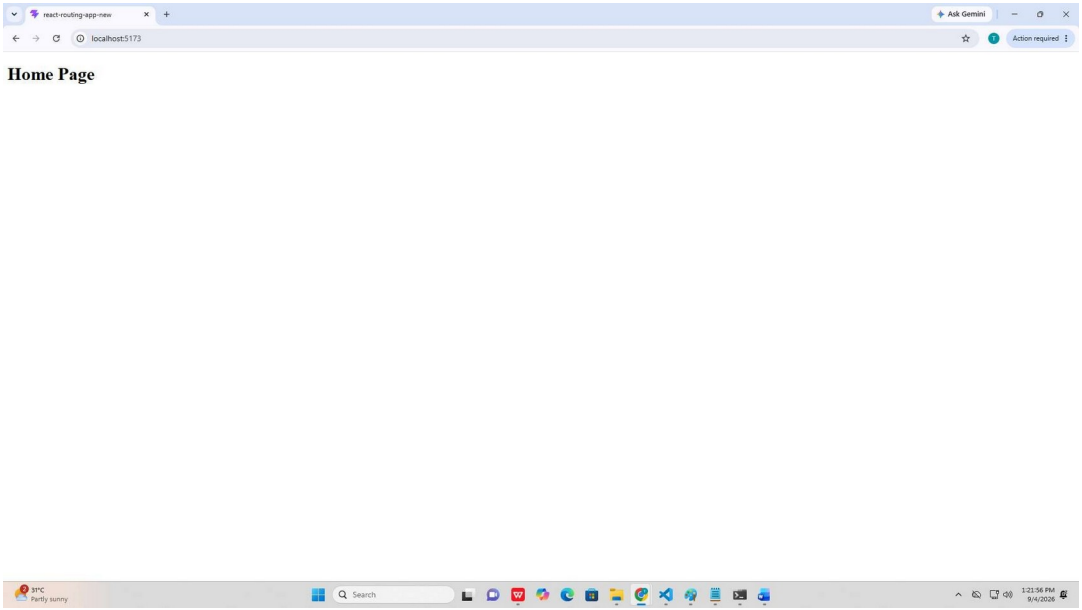
Step 4: Run the Application

In the terminal:

```
npm run dev
```

Open the URL provided by Vite, usually: http://localhost:5173/

Output:



5(b): Create multiple pages and implement navigation between them.

To create multiple pages in a React application and implement navigation between them using React Router, we will create three pages: Home, About and Contact.

Step 1: Create Pages Folder

Inside the src folder, create a folder named: pages. Create these three files inside it:

```
src
└── pages
    ├── Home.jsx
    ├── About.jsx
    └── Contact.jsx
```

Step 2: Create Home Page

Open Home.jsx and write:

```
function Home() {
  return (
    <div>
      <h1>Home Page</h1>
      <p>Welcome to the Home Page.</p>
    </div>
  );
}
export default Home;
```

Step 3: Create About Page

Open About.jsx:

```
function About() {
```

```
  return (
    <div>
      <h1>About Page</h1>
      <p>This is the About Page.</p>
    </div>
  );
}
export default About;
```

Step 4: Create Contact Page

Open Contact.jsx:

```
function Contact() {
  return (
    <div>
      <h1>Contact Page</h1>
      <p>Contact us at: example@gmail.com</p>
    </div>
  );
}
export default Contact;
```

Step 5: Configure Routes

Open: /src/App.jsx

```
import { Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  );
}

export default App;
```

Output:







5(c): Use Link and Routes to move between pages.

Step 1: Create the three pages under /src/pages: Home, About and Contact (as in 5(b)).

Step 2: Create Navbar.jsx under /src/components

```
import { Link } from "react-router-dom";

function Navbar() {
  return (
    <nav>
      <Link to="/">Home</Link> {" | "}
      <Link to="/about">About</Link> {" | "}
      <Link to="/contact">Contact</Link>
    </nav>
  );
}

export default Navbar;
```

Step 3: Edit App.jsx

```
import { Routes, Route } from "react-router-dom";
import Navbar from "./components/Navbar";
import Home from "./Pages/Home";
import About from "./Pages/About";
import Contact from "./Pages/Contact";

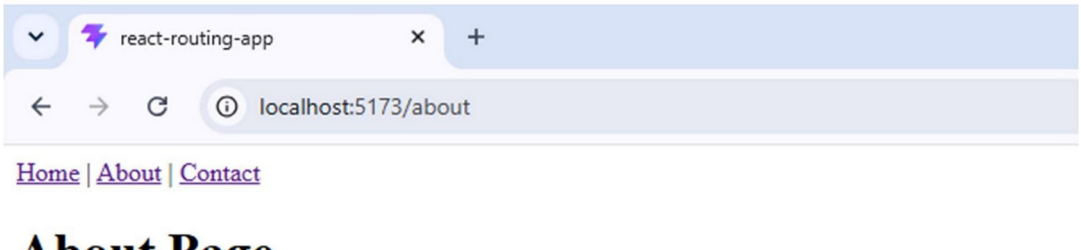
function App() {
  return (
    <>
```

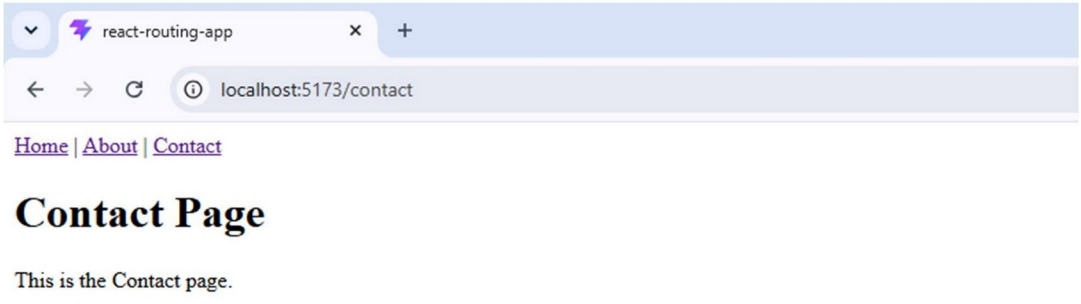
```
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </>
  );
}

export default App;
```

Output:







5(d): Fetch sample data from a public API using Axios.

Step 1: Create ApiData.jsx inside src/components

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function ApiData() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.log("Error fetching data:", error);
      });
  }, []);

  return (
    <div>
      <h2>Users Data</h2>
      <ul>
        {users.map((user) => (
          <li key={user.id}>
            {user.name} - {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default ApiData;
```

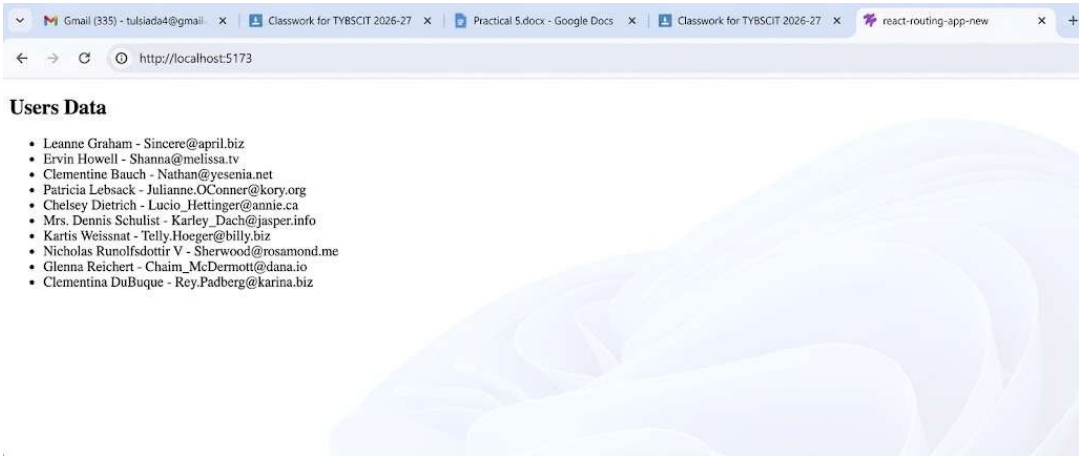
Step 2: Edit App.jsx

```
import ApiData from "./components/ApiData";

function App() {
  return (
    <div>
      <ApiData />
    </div>
  );
}

export default App;
```

Output:



5(e): Display fetched data in table or list format.

Step 1: Create ApiData.jsx inside src/components

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function ApiData() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.log("Error fetching data:", error);
      });
  }, []);

  return (
    <div>
      <h2>User Details</h2>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {users.map((user) => (
            <tr key={user.id}>
```

```
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.username}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default ApiData;
```

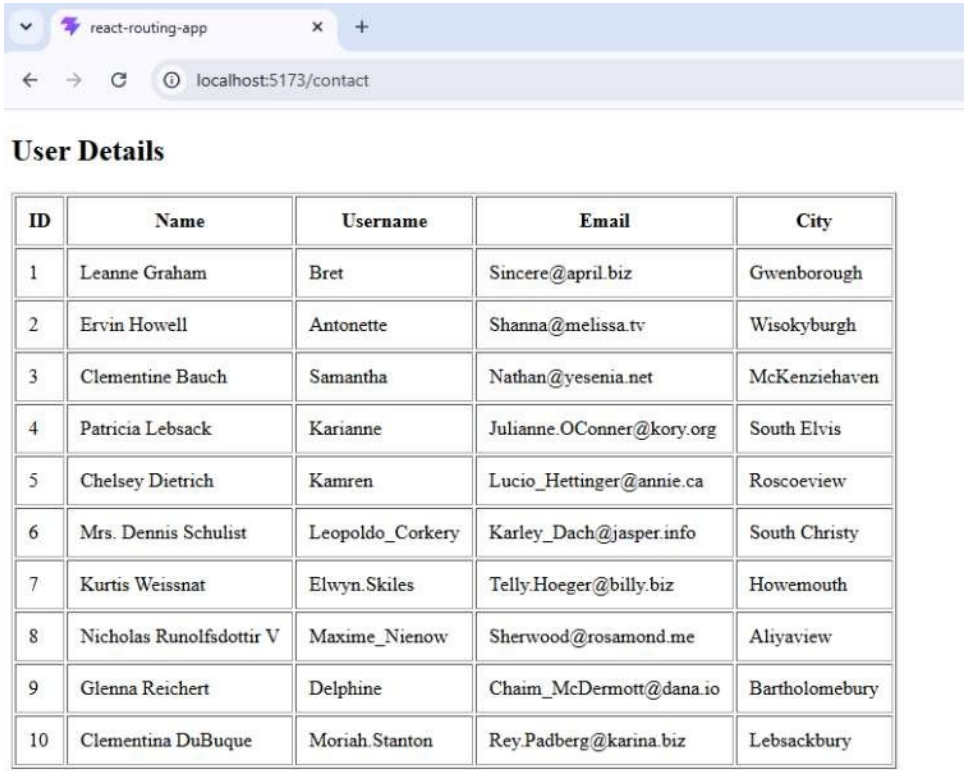
Step 2: Edit App.jsx

```
import ApiData from "./components/ApiData";

function App() {
  return (
    <div>
      <ApiData />
    </div>
  );
}

export default App;
```

Output:



ID	Name	Username	Email	City
1	Leanne Graham	Bret	Sincere@april.biz	Gwenborough
2	Ervin Howell	Antonette	Shanna@melissa.tv	Wisokyburgh
3	Clementine Bauch	Samantha	Nathan@yesenia.net	McKenziehaven
4	Patricia Lebsack	Karianne	Julianne.OConner@kory.org	South Elvis
5	Chelsey Dietrich	Kamren	Lucio_Hettinger@annie.ca	Roscoeview
6	Mrs. Dennis Schulist	Leopoldo_Corkery	Karley_Dach@jasper.info	South Christy
7	Kurtis Weissnat	Elwyn.Skiles	Telly.Hoeger@billy.biz	Howemouth
8	Nicholas Runolfsdottir V	Maxime_Nienow	Sherwood@rosamond.me	Aliyaview
9	Glenna Reichert	Delphine	Chaim_McDermott@dana.io	Bartholomebury
10	Clementina DuBuque	Moriah.Stanton	Rey.Padberg@karina.biz	Lebsackbury

5(f): Implement search or filter functionality on displayed data.

Step 1: Create ApiData.jsx inside src/components

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function ApiData() {
  const [users, setUsers] = useState([]);
  const [search, setSearch] = useState("");

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.log("Error fetching data:", error);
      });
  }, []);

  const filteredUsers = users.filter((user) =>
    user.name.toLowerCase().includes(search.toLowerCase()) ||
    user.username.toLowerCase().includes(search.toLowerCase()) ||
    user.email.toLowerCase().includes(search.toLowerCase())
```

```
  );

  return (
    <div>
      <h2>User Details</h2>
      <input
        type="text"
        placeholder="Search user..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
      <br />
      <br />
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.username}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default ApiData;
```

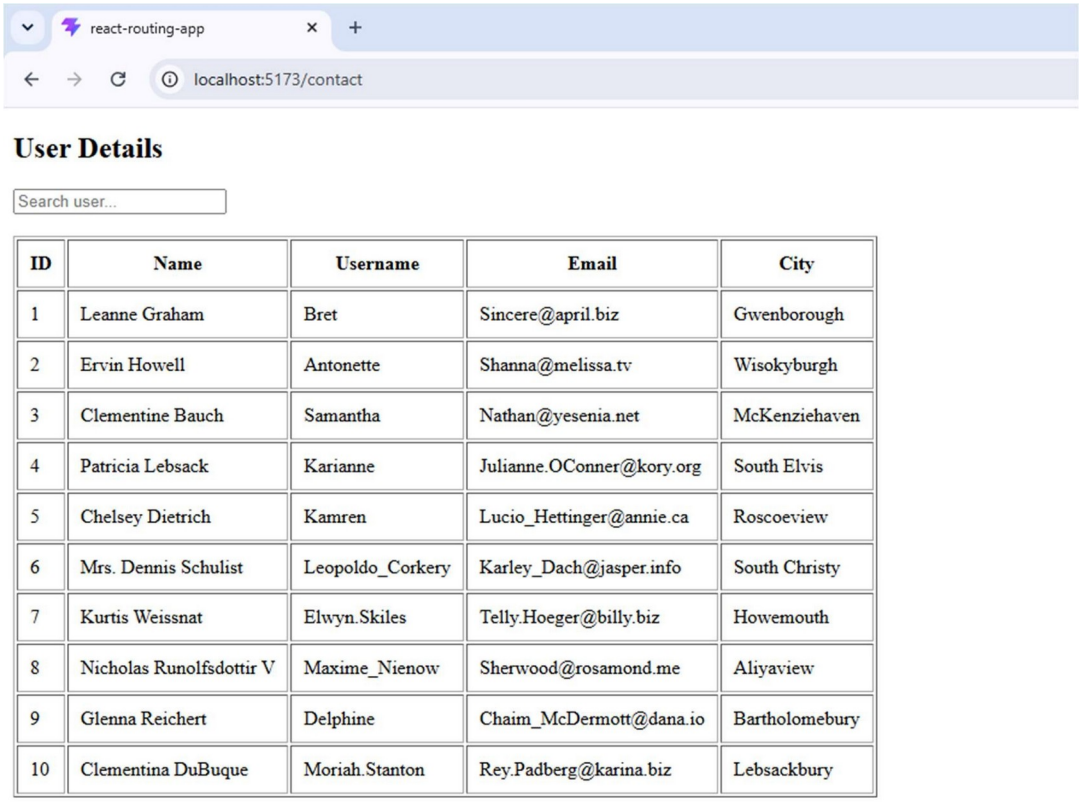
Step 2: Edit App.jsx

```
import ApiData from "./components/ApiData";

function App() {
  return (
    <div>
      <ApiData />
    </div>
  );
}

export default App;
```

Output:



ID	Name	Username	Email	City
1	Leanne Graham	Bret	Sincere@april.biz	Gwenborough
2	Ervin Howell	Antonette	Shanna@melissa.tv	Wisokyburgh
3	Clementine Bauch	Samantha	Nathan@yesenia.net	McKenziehaven
4	Patricia Lebsack	Karianne	Julianne.OConner@kory.org	South Elvis
5	Chelsey Dietrich	Kamren	Lucio_Hettinger@annie.ca	Roscoeview
6	Mrs. Dennis Schulist	Leopoldo_Corkery	Karley_Dach@jasper.info	South Christy
7	Kurtis Weissnat	Elwyn.Skiles	Telly.Hoeger@billy.biz	Howemouth
8	Nicholas Runolfsdottir V	Maxime_Nienow	Sherwood@rosamond.me	Aliyaview
9	Glenna Reichert	Delphine	Chaim_McDermott@dana.io	Bartholomebury
10	Clementina DuBuque	Moriah.Stanton	Rey.Padberg@karina.biz	Lebsackbury

PRACTICAL – 06

Node.js and Express Fundamentals

6(a): Install Express and create a simple server

Step 1: Create a project folder named “practical6”

Step 2: Open Terminal and Run `npm init -y`

```
C:\Users\Sunidhi>cd /

C:\>d:

D:\>cd MERN

D:\MERN>mkdir practical6

D:\MERN>cd practical6

D:\MERN\practical6>npm init -y
Wrote to D:\MERN\practical6\package.json:

{
  "name": "practical6",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Then install Express: `npm install express`

```
D:\MERN\practical6>npm install express

added 68 packages, and audited 69 packages in 2s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Step 3: Create a file named server.js inside the folder “practical6”

Open the file in VS Code and write the following code

```
const express = require("express");

const app = express();
const PORT = 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 4: Run the server

```
D:\MERN\practical6>node server.js
Server running on port 5000
```

6(b): Display a message on the browser using the server response

Add the following route to server.js:

```
const express = require("express");

const app = express();
const PORT = 5000;

app.get("/", (req, res) => {
  res.send("Welcome to MERN Stack Development");
});

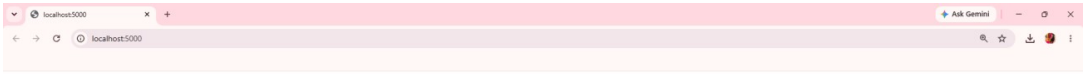
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Run the server

```
D:\MERN\practical6>node server.js
Server running on port 5000
```

OUTPUT

Open the browser: http://localhost:5000



Welcome to MERN Stack Development

6(c): Create different routes in Express

Add two more routes

```
const express = require("express");

const app = express();
const PORT = 5000;

app.get("/", (req, res) => {
  res.send("Welcome to MERN Stack Development");
});

app.get("/about", (req, res) => {
  res.send("This is the About Page");
});

app.get("/students", (req, res) => {
  res.send("Student Management Page");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Run the server

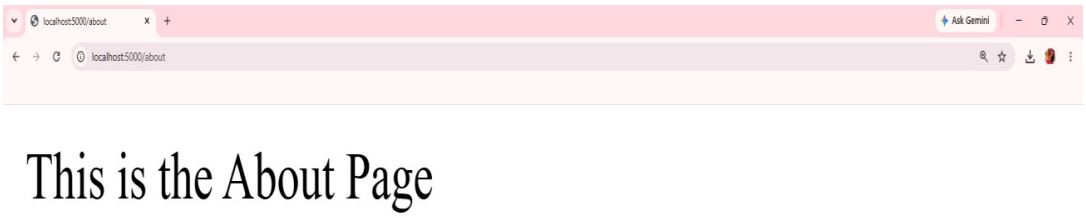
```
D:\MERN\practical6>node server.js
Server running on port 5000
```

OUTPUT

Open the browser: http://localhost:5000



Open the browser: http://localhost:5000/about



Open the browser: http://localhost:5000/students



6(d): Send a JSON response from the server

Add the following code to server.js and save

```
const express = require("express");

const app = express();
const PORT = 5000;

app.get("/", (req, res) => {
  res.send("Welcome to MERN Stack Development");
});
app.get("/api/student", (req, res) => {
  const student = {
    id: 101,
    name: "Rahul",
    course: "MERN Stack",
    marks: 85
  };

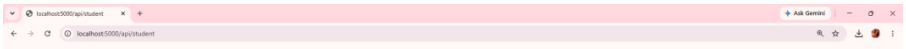
  res.json(student);
});
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Run the server

```
D:\MERN\practical6>node server.js
Server running on port 5000
```

OUTPUT

Open the browser: http://localhost:5000/api/student



Pretty-print ☑


```
{
  "id": 101,
  "name": "Rahul",
  "course": "MERN Stack",
  "marks": 85
}
```

6(e): Test routes using Postman

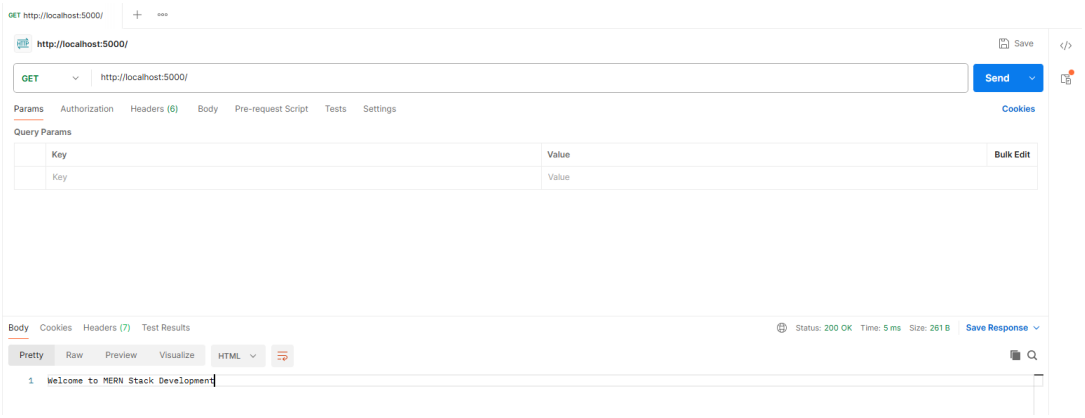
Step 1: Install Postman

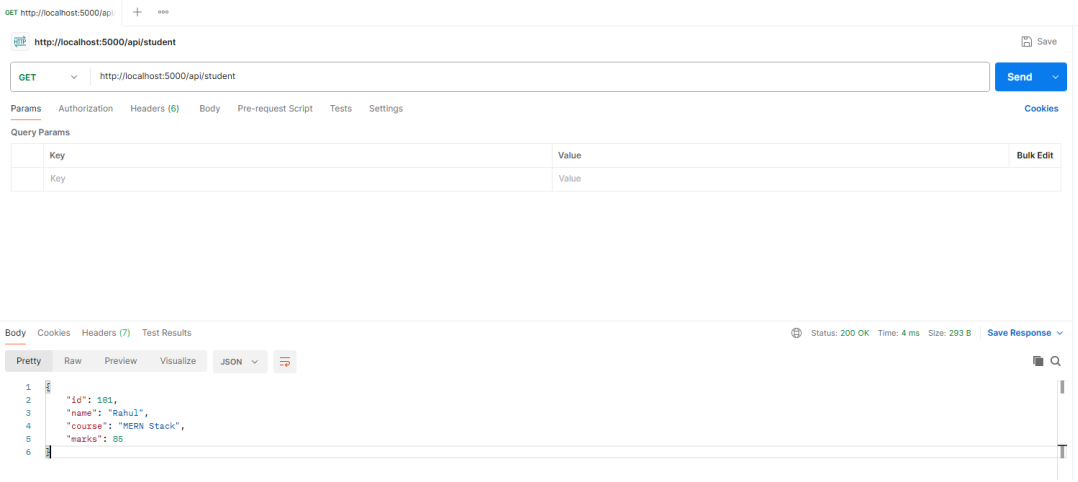
Step 2:

1. Open Postman
2. Select GET
3. Enter the URL
4. Click send
5. Observe the response

Method	URL	Response
GET	http://localhost:5000/	Welcome message
GET	http://localhost:5000/about	About Page
GET	http://localhost:5000/students	Student Page
GET	http://localhost:5000/api/student	JSON data

OUTPUT




GET http://localhost:5000/api
http://localhost:5000/api/student
Save
GET
http://localhost:5000/api/student
Send
Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Bulk Edit
Key
Value
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 4 ms
Size: 293 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"id": 101,
"name": "Rahul",
"course": "MERN Stack",
"marks": 85
}

6(f): Run the server and observe the output in the console

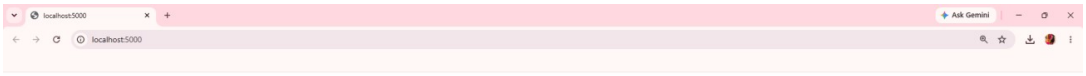
```
const express = require("express");
const app = express();
const PORT = 5000;
// Home route
app.get("/", (req, res) => {
  console.log("Home route requested");
  res.send("Welcome to MERN Stack Development");
});
// About route
app.get("/about", (req, res) => {
  console.log("About route requested");
  res.send("This is the About Page");
});
// Students route
app.get("/students", (req, res) => {
  console.log("Students route requested");
  res.send("Student Management Page");
});
// JSON API
app.get("/api/student", (req, res) => {
  console.log("Student API requested");

  const student = {
    id: 101,
    name: "Rahul",
    course: "MERN Stack",
    marks: 85
  };

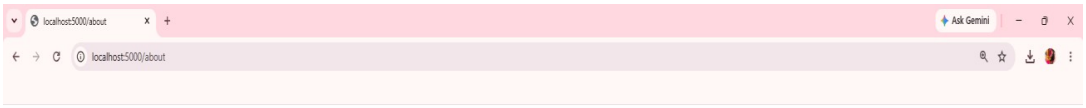
  res.json(student);
});
// Start server
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

OUTPUT

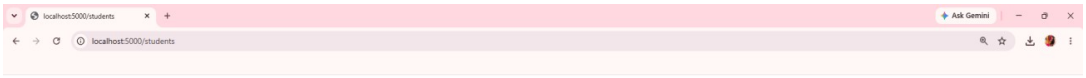
```
D:\MERN\practical6>node server.js
Server running on port 5000
Home route requested
About route requested
Students route requested
Student API requested
```



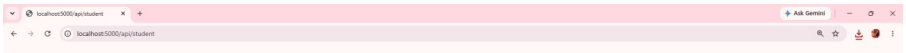
Welcome to MERN Stack Development



This is the About Page



Student Management Page



Pretty-print ☑


```
{
  "id": 101,
  "name": "Rahul",
  "course": "MERN Stack",
  "marks": 85
}
```

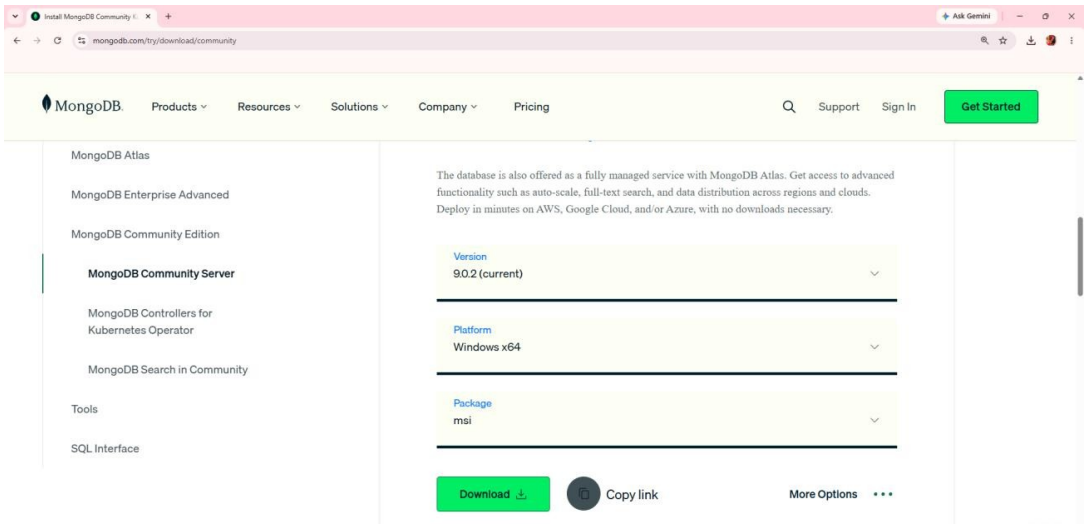
PRACTICAL – 07

MongoDB Basics

7(a): Install and start MongoDB

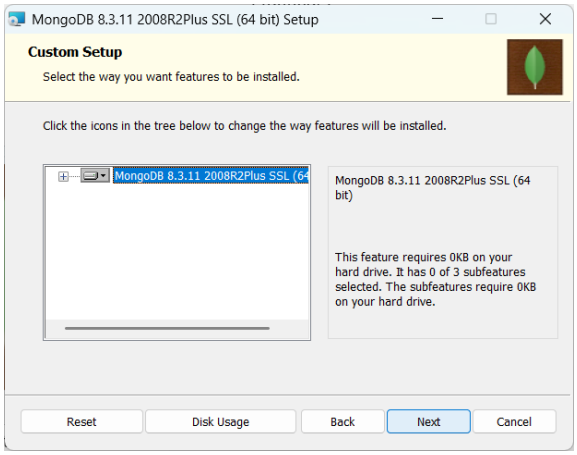
1. Download mongodb from official website

   https://www.mongodb.com/try/download/community



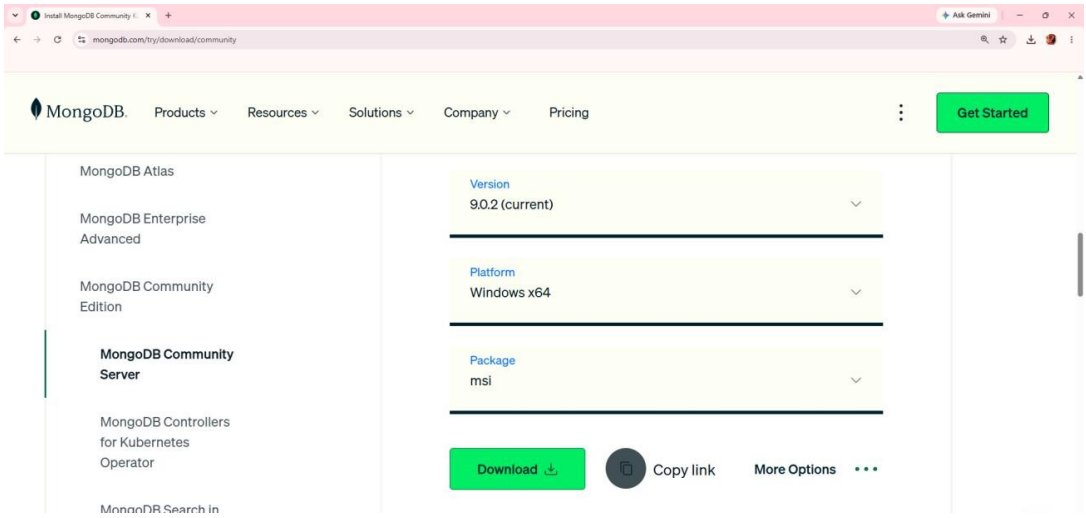
2. Install mongodb





3. Download mongodb shell from the following link

   https://www.mongodb.com/try/download/shell

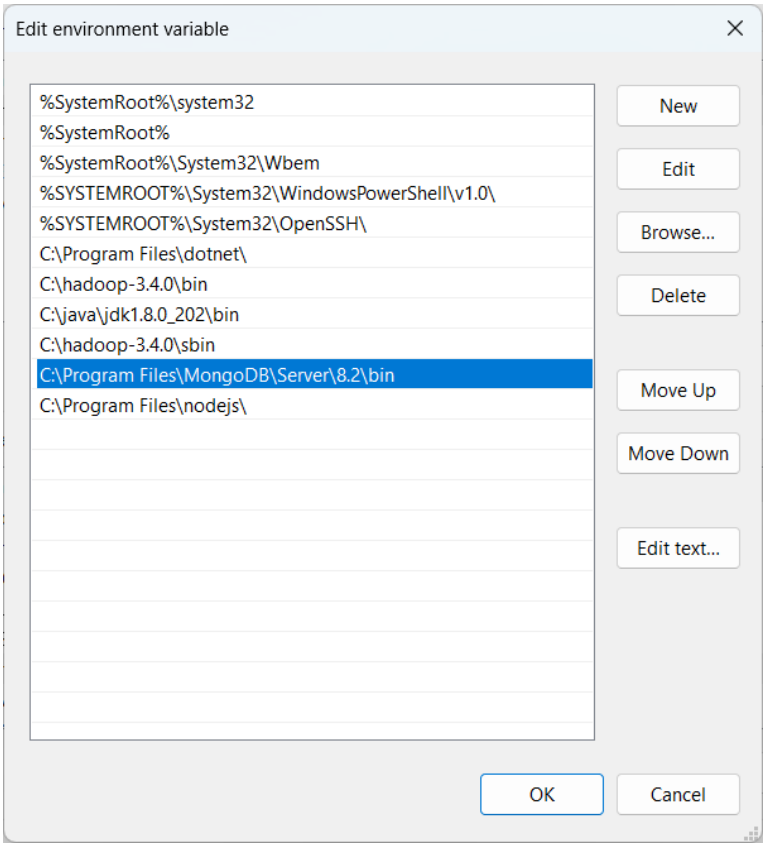


4. Install mongosh



5. Go to system environmental variables and set the path

   C:\Program Files\MongoDB\Server\8.3\bin



6. Check the installation

```
Microsoft Windows [Version 10.0.22621.3296]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo>mongosh
Current Mongosh Log ID: 6ac5c9af19064b1ded628c9f
Connecting to:          mongodb://127.0.0.1:27017/?
directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB:          8.2.3
Using Mongosh:          2.6.0
mongosh 2.12.0 is available for download:
https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-09-26T12:09:29.254+05:30: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

test>
```

7(b): Create a database and collection

1. Open the command prompt and type “mongosh”
2. Create/Select a Database: Use command “use mongoDB”
3. Create a Collection

   db.createCollection("student_new")

```
Microsoft Windows [Version 10.0.26200.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sunidhi>mongosh
Current Mongosh Log ID: 6ab4e2506cf5f8d46ea4294b
Connecting to:          mongodb://127.0.0.1:27017/?
directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:          8.3.11
Using Mongosh:          2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-09-24T12:21:39.443+05:30: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

test> use mongoDB
switched to db mongoDB
mongoDB> db.createCollection("student_new")
{ ok: 1 }
mongoDB>
```

4. Check the Collection: “show collections”

7(c): Insert records in MongoDB

Make sure the database and collection are actually created, insert a student:

1. Insert one record at time in the table

```
------
   The server generated these startup warnings when booting
   2026-09-24T12:21:39.443+05:30: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

test> use mongoDB
switched to db mongoDB
mongoDB> show collections
student_new
mongoDB> db.student_new.insertOne({
|     name: "Rahul",
|     course: "MERN",
|     marks: 85
| })
{
  acknowledged: true,
  insertedId: ObjectId('6ab4e5424662df3569dfc0da')
}
mongoDB> db.student_new.find()
[
  {
    _id: ObjectId('6ab4e5424662df3569dfc0da'),
    name: 'Rahul',
    course: 'MERN',
    marks: 85
  }
]
mongoDB>
```

2. Insert many records at a time in the table

```
mongoDB> db.students.insertMany([
|     {
|         name: "Rahul",
|         course: "MERN",
|         marks: 85
|     },
|     {
|         name: "Priya",
|         course: "React",
|         marks: 90
|     },
|     {
|         name: "Amit",
|         course: "Node.js",
|         marks: 78
|     },
|     {
|         name: "Sneha",
```

```
|         course: "MongoDB",
|         marks: 88
|     }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ab4e6d84662df3569dfc0db'),
    '1': ObjectId('6ab4e6d84662df3569dfc0dc'),
    '2': ObjectId('6ab4e6d84662df3569dfc0dd'),
    '3': ObjectId('6ab4e6d84662df3569dfc0de')
  }
}
```

7(d): View records from the collection

```
mongoDB> db.students.find()
[
  {
    _id: ObjectId('6ab4e6d84662df3569dfc0db'),
    name: 'Rahul',
    course: 'MERN',
    marks: 85
  },
  {
    _id: ObjectId('6ab4e6d84662df3569dfc0dc'),
    name: 'Priya',
    course: 'React',
    marks: 90
  },
  {
    _id: ObjectId('6ab4e6d84662df3569dfc0dd'),
    name: 'Amit',
    course: 'Node.js',
    marks: 78
  },
  {
    _id: ObjectId('6ab4e6d84662df3569dfc0de'),
    name: 'Sneha',
    course: 'MongoDB',
    marks: 88
  }
]
mongoDB>
```

7(e): Update and delete records

1. Update one record and see the result

```
    marks: 88
  }
]
mongoDB> db.student_new.updateOne(
|     { name: "Rahul" },
|     { $set: { marks: 90 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
mongoDB> db.student_new.find({ name: "Rahul" })
[
  {
    _id: ObjectId('6ab4e6d84662df3569dfc0db'),
    name: 'Rahul',
    course: 'MERN',
    marks: 85
  }
]
mongoDB>
```

2. Update multiple records

   Increase the marks of all students by 5:

```
]
mongoDB> db.student_new.updateMany(
|     {},
|     { $inc: { marks: 5 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 5,
  modifiedCount: 5,
  upsertedCount: 0
}
mongoDB> db.student_new.find()
[
  {
    _id: ObjectId('6ab4e5424662df3569dfc0da'),
    name: 'Rahul',
    course: 'MERN',
    marks: 100
  },
  {
    _id: ObjectId('6ab51d4c383a4f769f6164d4'),
    name: 'Rahul',
```

```
  course: 'MERN',
  marks: 90
},
{
  _id: ObjectId('6ab51d4c383a4f769f6164d5'),
  name: 'Priya',
  course: 'React',
  marks: 95
},
{
  _id: ObjectId('6ab51d4c383a4f769f6164d6'),
  name: 'Amit',
  course: 'Node.js',
  marks: 83
},
```

3. Delete one record: Delete the student named Amit:

```
  }
]
mongoDB> db.student_new.deleteOne({ name: "Amit" })
{ acknowledged: true, deletedCount: 1 }
mongoDB>
```

4. Delete multiple records: delete all students whose marks are below 95:

```
mongoDB> db.student_new.find()
[
  {
    _id: ObjectId('6ab4e5424662df3569dfc0da'),
    name: 'Rahul',
    course: 'MERN',
    marks: 100
  },
  {
    _id: ObjectId('6ab51d4c383a4f769f6164d4'),
    name: 'Rahul',
    course: 'MERN',
    marks: 90
  },
  {
    _id: ObjectId('6ab51d4c383a4f769f6164d5'),
    name: 'Priya',
    course: 'React',
    marks: 95
  },
  {
    _id: ObjectId('6ab51d4c383a4f769f6164d7'),
    name: 'Sneha',
    course: 'MongoDB',
    marks: 93
  }
]
mongoDB> db.student_new.deleteMany({ marks: { $lt: 95 } })
{ acknowledged: true, deletedCount: 2 }
mongoDB>
```

7(f): Connect MongoDB with Node.js

We will use Mongoose to connect Node.js with the MongoDB database mongoDB

1. Open your Node.js project: Open your project folder in VS Code
2. Create a file : “server.js”
3. Initialize the Node.js project

   Open the terminal and execute the command: npm init -y

```
C:\Users\Sunidhi>d:

D:\>cd MERN

D:\MERN>cd "MERN Project"

D:\MERN\MERN Project>cd Practical7

D:\MERN\MERN Project\Practical7>npm init -y
Wrote to D:\MERN\MERN Project\Practical7\package.json:

{
  "name": "practical7",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

D:\MERN\MERN Project\Practical7>
```

4. Install Mongoose: Mongoose is used to connect Node.js applications with MongoDB

```
D:\MERN\MERN Project\Practical7>npm install mongoose

added 19 packages, and audited 20 packages in 4s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\MERN\MERN Project\Practical7>
```

5. Write the connection program : In server.js

```
const mongoose = require("mongoose");

mongoose.connect("mongodb://127.0.0.1:27017/mongoDB")
  .then(() => {
    console.log("MongoDB Connected Successfully");
  })
  .catch((error) => {
    console.log("Connection Error:", error);
  });
```

```
D:\MERN\MERN Project\Practical7>node server.js
MongoDB Connected Successfully
```

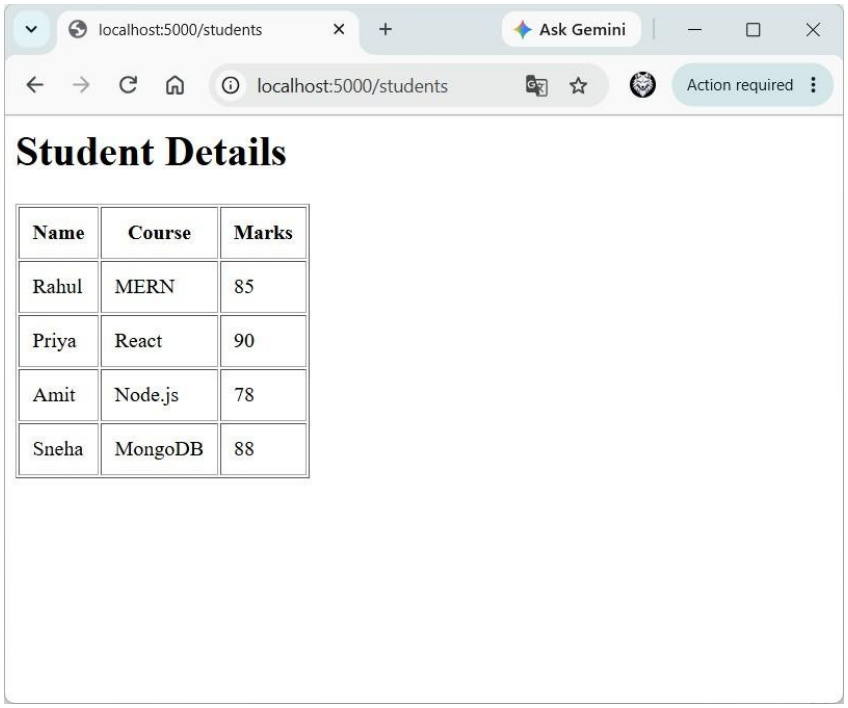
6. Get the data from MongoDB database and display on the web page, write the following code in “server.js” and run the command “node server.js”

Code:

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();
// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/mongoDB")
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log("Connection Error:", err));
// Student schema
const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  marks: Number
});
// Use existing collection: students
const Student = mongoose.model("Student", studentSchema, "students");
// Display MongoDB data in a web page
app.get("/students", async (req, res) => {
  const students = await Student.find();
  let table = `
    <h1>Student Details</h1>
    <table border="1" cellpadding="10">
      <tr>
        <th>Name</th>
        <th>Course</th>
        <th>Marks</th>
      </tr>
  `;
  students.forEach((student) => {
    table += `
      <tr>
        <td>${student.name}</td>
        <td>${student.course}</td>
        <td>${student.marks}</td>
      </tr>
    `;
  });
  table += `
    </table>
  `;
  res.send(table);
});
// Start server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

```
D:\MERN\MERN Project\Practical7>node server.js
Server running on port 5000
MongoDB Connected
```

7. Open the browser and type the url: http://localhost:5000/students


localhost:5000/students
Ask Gemini
localhost:5000/students
Action required
Student Details
Name
Course
Marks
Rahul
MERN
85
Priya
React
90
Amit
Node.js
78
Sneha
MongoDB
88

PRACTICAL – 08

API Creation and Integration

8(a): Create a simple REST API using Express

1. Create the backend: Create a folder “Practical8/server”
2. Open the server folder in the terminal and initialize npm

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sunidhi>d:

D:\>cd MERN

D:\MERN>cd "Practical 8"

D:\MERN\Practical 8>cd server

D:\MERN\Practical 8\server>npm init -y
Wrote to D:\MERN\Practical 8\server\package.json:

{
  "name": "server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

3. Install the following:
    - Express is used to create the backend/server and APIs.
    - Mongoose connects your Node.js/Express application to MongoDB.
    - CORS allows your React frontend and Express backend to communicate when they run on different ports.

```
D:\MERN\Practical 8\server>
D:\MERN\Practical 8\server>npm install express mongoose cors

added 88 packages, and audited 89 packages in 6s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\MERN\Practical 8\server>
```

4. Create “server.js”

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();

app.use(cors());
app.use(express.json());

// Connect MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/mongoDB")
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log("MongoDB Error:", err));

// Student schema
const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  marks: Number
});

// Use existing collection
const Student = mongoose.model(
  "Student",
  studentSchema,
  "student_new"
);

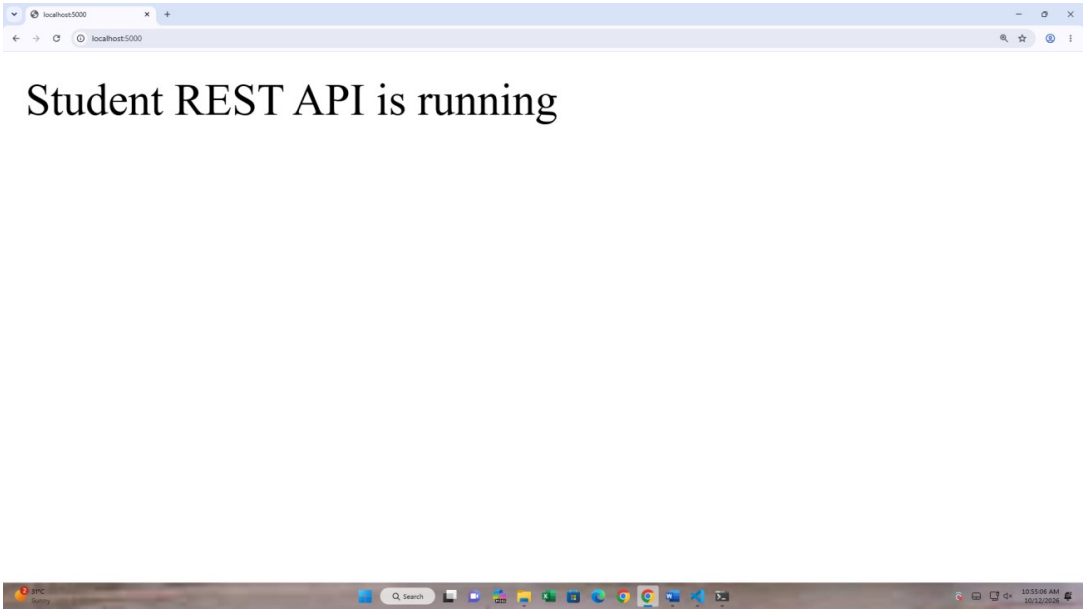
// Simple REST API
app.get("/", (req, res) => {
  res.send("Student REST API is running");
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

5. Run the server

```
D:\MERN\Practical 8\server>node server.js
Server running on port 5000
MongoDB Connected
```

OUTPUT



8(b): Implement a GET API to retrieve data

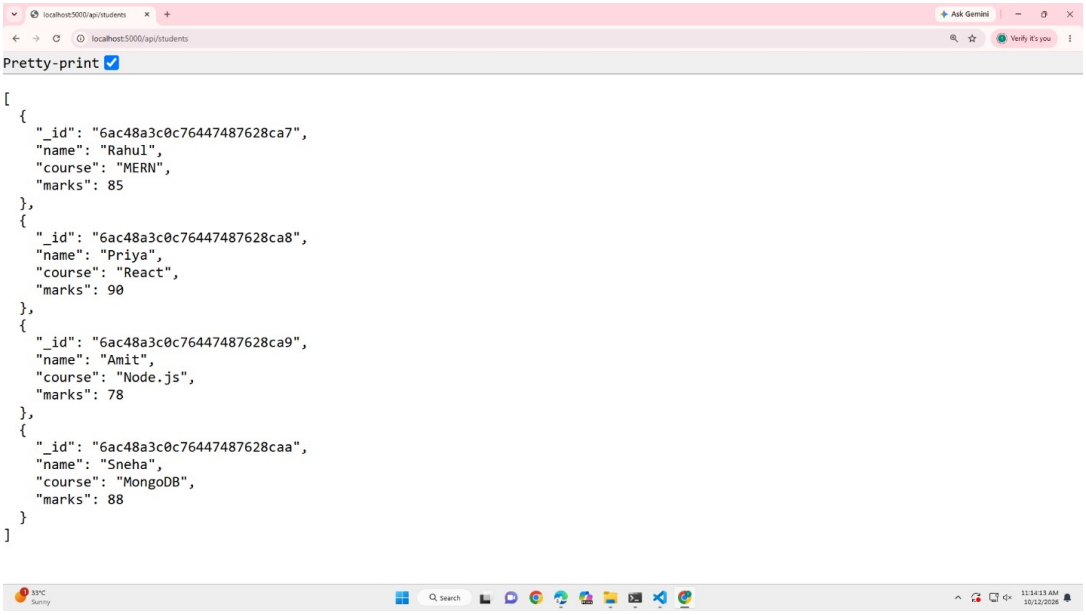
1. Add the following route to server.js and save the file

```
app.get("/api/students", async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  }
  catch (error) {
    res.status(500).json({
      error: error.message
    });
  }
});
```

2. Open the browser and paste the link http://localhost:5000/api/students

   You will get the records from MongoDB → students

OUTPUT


localhost:5000/api/students
Ask Gemini
localhost:5000/api/students
Verify it's you
Pretty-print
[
{
"_id": "6ac48a3c0c76447487628ca7",
"name": "Rahul",
"course": "MERN",
"marks": 85
},
{
"_id": "6ac48a3c0c76447487628ca8",
"name": "Priya",
"course": "React",
"marks": 90
},
{
"_id": "6ac48a3c0c76447487628ca9",
"name": "Amit",
"course": "Node.js",
"marks": 78
},
{
"_id": "6ac48a3c0c76447487628caa",
"name": "Sneha",
"course": "MongoDB",
"marks": 88
}
]
Search

8(c): Fetch data from the API in a React application

1. Create the React application from the practical8 folder:

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sunidhi>d:

D:\>cd MERN

D:\MERN>cd "Practical 8"

D:\MERN\Practical 8>npm create vite@latest client

> npx
> create-vite client

|
o  Select a framework:
|  React
|
o  Select a variant:
|  JavaScript
|
o  Which linter to use?
|  ESLint
|
*  Install with npm and start now?
|  > Yes /   No
```

2. Get into the client folder and run the application

   Open the App.jsx in visual studio code and replace the code with following code

```
import { useEffect, useState } from "react";

function App() {
  const [students, setStudents] = useState([]);

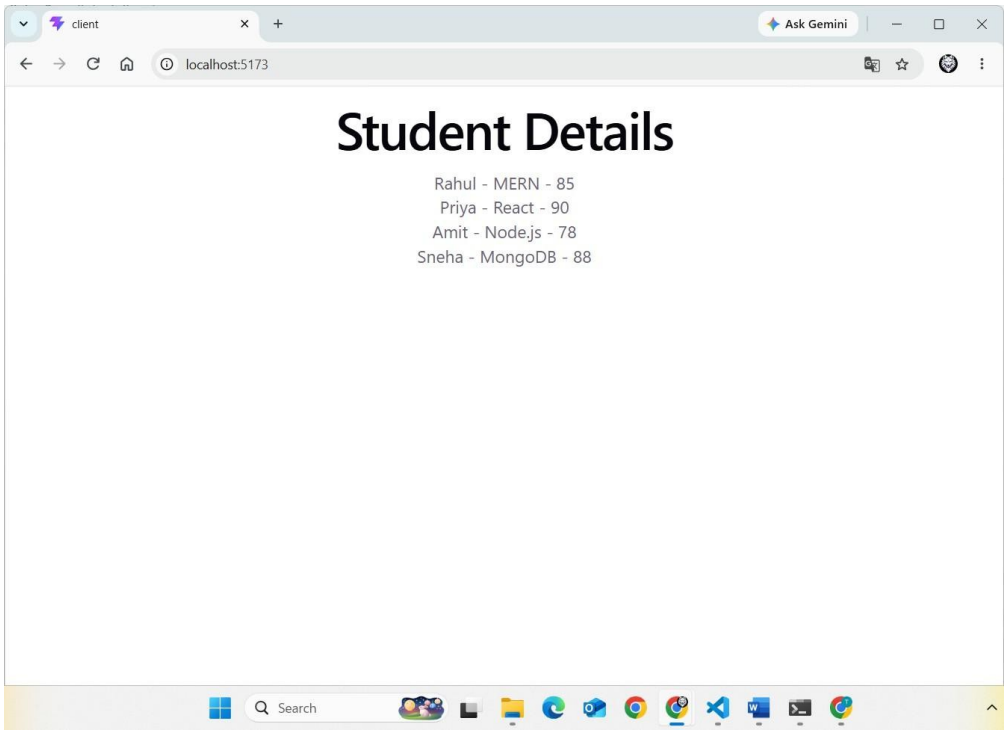
  useEffect(() => {
    fetch("http://localhost:5000/api/students")
      .then(response => response.json())
      .then(data => setStudents(data));
  }, []);

  return (
    <div>
      <h1>Student Details</h1>
      {students.map((student) => (
        <p key={student._id}>
          {student.name} - {student.course} - {student.marks}
        </p>
      ))}
    </div>
  );
}
```

```
export default App;
```

3. Run the code “npm run dev” and browser. Now the data will be fetched from the database and display on the browser

OUTPUT



8(d): Send data from React to the server using a POST request

1. Code for server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
// Middleware
app.use(cors());
app.use(express.json());
// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/mongoDB")
  .then(() => {
    console.log("MongoDB Connected Successfully");
  })
  .catch((error) => {
    console.log("MongoDB Error:", error);
  });
// Student Schema
const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  marks: Number
});
// Use the existing "students" collection
const Student = mongoose.model(
  "Student",
  studentSchema,
  "students"
);
// GET API
app.get("/api/students", async (req, res) => {
  try {
    const students = await Student.find();
    console.log("Students fetched:", students);
    res.json(students);
  } catch (error) {
    console.log("Error:", error);
    res.status(500).json({
      error: error.message
    });
  }
});
// POST API
app.post("/api/students", async (req, res) => {
  try {
    const student = new Student({
      name: req.body.name,
      course: req.body.course,
      marks: req.body.marks
    });
    await student.save();
    console.log("Student added:", student);
    res.json({
      message: "Student added successfully",
```

```
      student: student
    });
  } catch (error) {
    res.status(500).json({
      error: error.message
    });
  }
});
// Start Server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

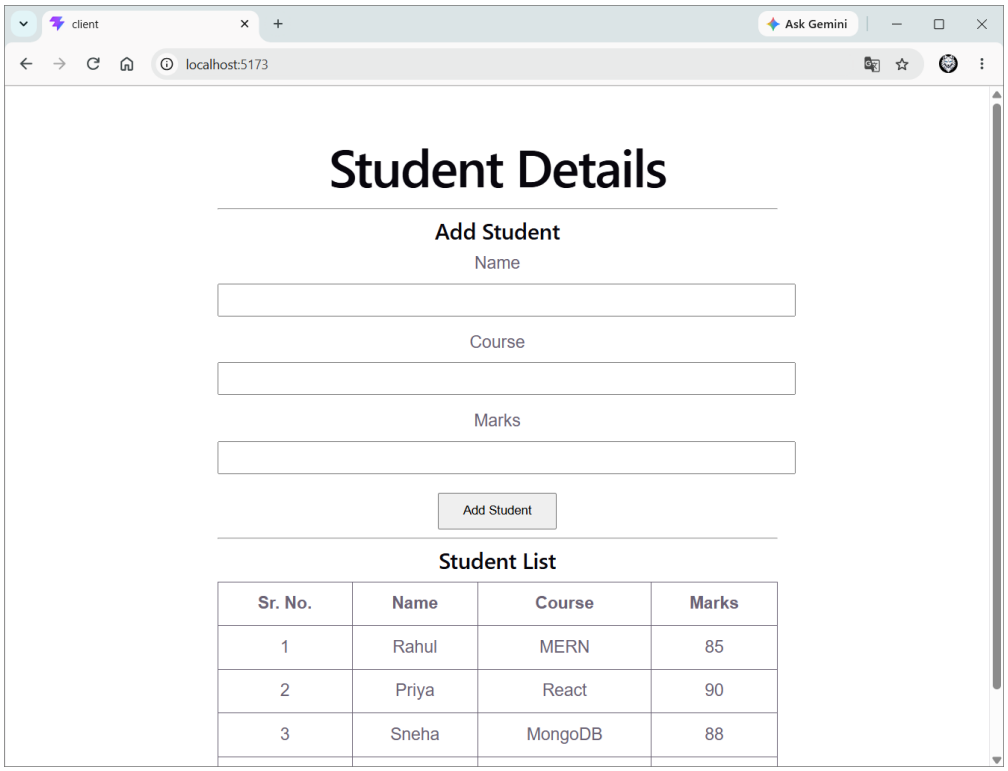
2. Code for App.jsx

```
import { useEffect, useState } from "react";
function App() {
  const [students, setStudents] = useState([]);
  const [name, setName] = useState("");
  const [course, setCourse] = useState("");
  const [marks, setMarks] = useState("");
  // Fetch students
  const getStudents = () => {
    fetch("http://localhost:5000/api/students")
      .then(response => response.json())
      .then(data => setStudents(data))
      .catch(error => console.log(error));
  };
  useEffect(() => {
    getStudents();
  }, []);
  // Add student
  const addStudent = () => {
    if (!name || !course || !marks) {
      alert("Please enter all details");
      return;
    }
    fetch("http://localhost:5000/api/students", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        name: name,
        course: course,
        marks: Number(marks)
      })
    })
      .then(response => response.json())
      .then(() => {
        setName("");
        setCourse("");
        setMarks("");
        getStudents();
      });
```

```
};
return (
  <div style={{
    width: "600px",
    margin: "40px auto",
    fontFamily: "Arial"
  }}>
    <h1 style={{ textAlign: "center" }}>
      Student Details
    </h1>
    <hr />
    <h2>Add Student</h2>
    <label>Name</label>
    <br />
    <input
      type="text"
      value={name}
      onChange={(e) => setName(e.target.value)}
      style={{
        width: "100%",
        padding: "8px",
        margin: "8px 0 15px"
      }}
    />
    <br />
    <label>Course</label>
    <br />
    <input
      type="text"
      value={course}
      onChange={(e) => setCourse(e.target.value)}
      style={{
        width: "100%",
        padding: "8px",
        margin: "8px 0 15px"
      }}
    />
    <br />
    <label>Marks</label>
    <br />
    <input
      type="number"
      value={marks}
      onChange={(e) => setMarks(e.target.value)}
      style={{
        width: "100%",
        padding: "8px",
        margin: "8px 0 20px"
      }}
    />
    <br />
    <button
      onClick={addStudent}
      style={{
        padding: "10px 25px",
        cursor: "pointer"
```

```
        }}
      >
        Add Student
      </button>
      <hr />
      <h2>Student List</h2>
      <table
        border="1"
        cellPadding="10"
        style={{
          width: "100%",
          borderCollapse: "collapse",
          textAlign: "center"
        }}
      >
        <thead>
          <tr>
            <th>Sr. No.</th>
            <th>Name</th>
            <th>Course</th>
            <th>Marks</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student, index) => (
            <tr key={student._id}>
              <td>{index + 1}</td>
              <td>{student.name}</td>
              <td>{student.course}</td>
              <td>{student.marks}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
export default App;
```

OUTPUT


client
Ask Gemini
localhost:5173
Student Details
Add Student
Name
Course
Marks
Add Student
Student List
Sr. No.
Name
Course
Marks
1
Rahul
MERN
85
2
Priya
React
90
3
Sneha
MongoDB
88

8(e): Store data in MongoDB

1. Use the existing setup

   Add POST API in server.js

```
app.post("/api/students", async (req, res) => {
  try {
    const student = new Student({
      name: req.body.name,
      course: req.body.course,
      marks: req.body.marks
    });
    await student.save();
    res.json({
      message: "Student data stored successfully",
      student: student
    });
  } catch (error) {
    res.status(500).json({
      error: error.message
    });
  }
});
```

2. How it works:

   When the user enters:

   Name : Rahul
   Course : MERN
   Marks : 85

   React sends:

```
{
  "name": "Rahul",
  "course": "MERN",
  "marks": 85
}
```

   The express server receives it:

   req.body

   Then creates a MongoDB document:

```
const student = new Student({
  name: req.body.name,
  course: req.body.course,
  marks: req.body.marks
});
```

Finally:

```
await student.save();
```

3. Verify in MongoDB

```
mongoDB> db.students.find()
[
  {
    _id: ObjectId('6ab4e6d84662df3569dfc0db'),
    name: 'Rahul',
    course: 'MERN',
    marks: 85
  },
  {
    _id: ObjectId('6ab4e6d84662df3569dfc0dc'),
    name: 'Priya',
    course: 'React',
    marks: 90
  },
  {
    _id: ObjectId('6ab4e6d84662df3569dfc0de'),
    name: 'Sneha',
    course: 'MongoDB',
    marks: 88
  },
  {
    _id: ObjectId('6ab656d8a2543c3ff031fc46'),
    name: 'Raj',
    course: 'BSCIT',
    marks: 450,
    __v: 0
  },
  {
    _id: ObjectId('6ab65a15254cdb03576e4206'),
    name: 'Varun',
```

8(f): Display stored data in React page

Add the following code to “App.jsx”

```
import { useEffect, useState } from "react";

function App() {
  const [students, setStudents] = useState([]);

  // Fetch data from server
  useEffect(() => {
    fetch("http://localhost:5000/api/students")
      .then(response => response.json())
      .then(data => {
        setStudents(data);
      })
      .catch(error => {
        console.log("Error:", error);
      });
  }, []);

  return (
    <div
      style={{
        width: "700px",
        margin: "40px auto",
        fontFamily: "Arial"
      }}
    >
      <h1 style={{ textAlign: "center" }}>
        Student Details
      </h1>

      <hr />

      <h2>Stored Student Data</h2>

      <table
        border="1"
        cellPadding="10"
        style={{
          width: "100%",
          borderCollapse: "collapse",
          textAlign: "center"
        }}
      >
        <thead>
          <tr>
            <th>Sr. No.</th>
            <th>Name</th>
            <th>Course</th>
            <th>Marks</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student, index) => (
```

```
            <tr key={student._id}>
              <td>{index + 1}</td>
              <td>{student.name}</td>
              <td>{student.course}</td>
              <td>{student.marks}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
export default App;
```

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
const PORT = 5000;

app.use(cors());
app.use(express.json());

// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/mongoDB")
  .then(() => console.log("MongoDB Connected Successfully"))
  .catch(error => console.log("MongoDB Error:", error.message));

// Student schema
const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  marks: Number
});

// Existing collection
const Student = mongoose.model(
  "Student",
  studentSchema,
  "students"
);

// Home route
app.get("/", (req, res) => {
  res.send("Student API is running");
});

// GET: Display stored student data
app.get("/api/students", async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (error) {
```

```
    res.status(500).json({ error: error.message });
  }
});

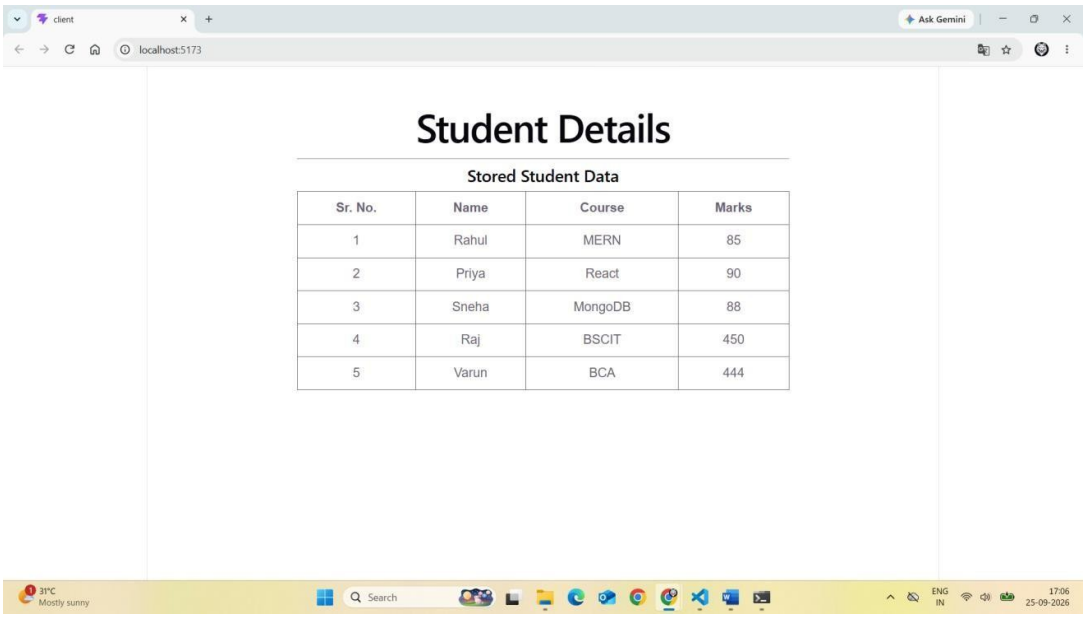
// POST: Store a new student
app.post("/api/students", async (req, res) => {
  try {
    const student = new Student({
      name: req.body.name,
      course: req.body.course,
      marks: req.body.marks
    });

    await student.save();

    res.status(201).json({
      message: "Student added successfully",
      student: student
    });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});

// Start server
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

OUTPUT



PRACTICAL – 09
User Data Management

9(a): Create an API to add user details using Express and MongoDB

Step 1: Create the project folder and initialize it using npm.

```
D:\>cd MERN
D:\MERN>cd prac9
D:\MERN\prac9> npm init -y
Wrote to D:\MERN\prac9\package.json:
{
  "name": "prac9",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Step 2: Install the required packages inside the server folder.

```
D:\MERN\Prac9\server>npm install express cors mongoose
added 88 packages, and audited 89 packages in 3s
30 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

Step 3: Create server.js and write the following code.

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/userdb")
  .then(() => console.log("MongoDB Connected"))
  .catch((error) => console.log(error));
// User Schema
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});
// User Model
const User = mongoose.model("User", userSchema);
// Add User API
```

```
app.post("/users", async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.json({
      message: "User added successfully",
      user: user
    });
  } catch (error) {
    res.status(500).json({
      error: error.message
    });
  }
});
// Start Server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

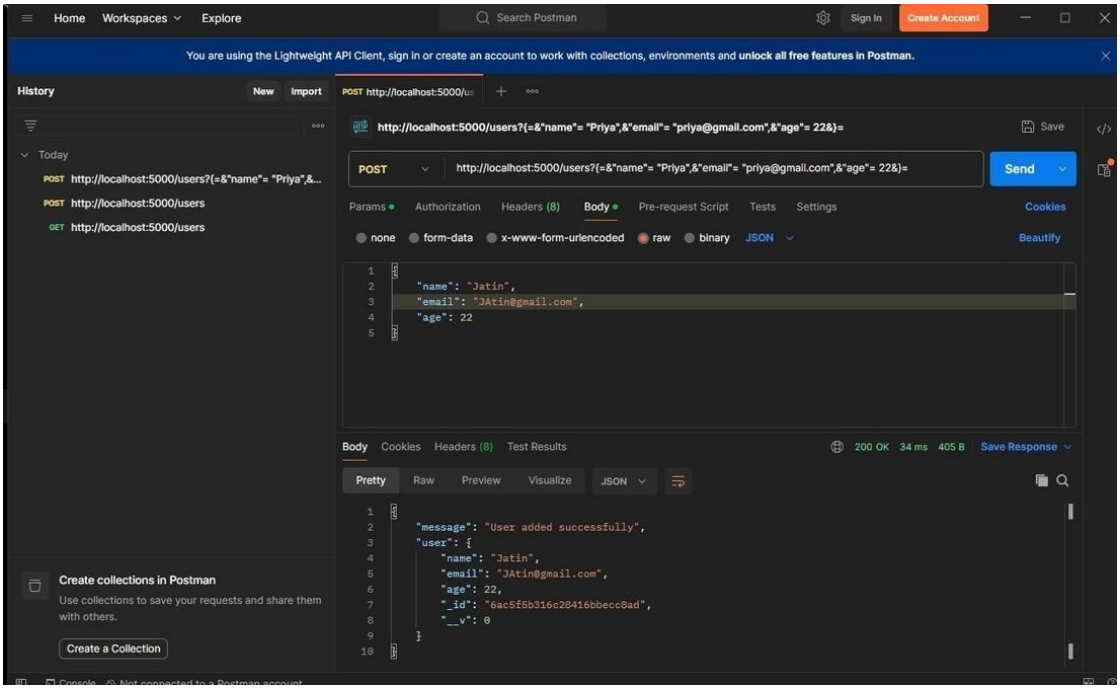
Step 4: Run the server using the terminal.

```
node server.js
```

```
D:\MERN\Prac9\server>node server.js
Server running on port 5000
MongoDB Connected
```

Step 5: Test the API using Postman. Send a POST request to http://localhost:5000/users with the user details (name, email, age) in the Body.

OUTPUT



Note: To change the user details, open the Body tab in Postman, edit the values and click Send.

9(b): Design a React form to send user data to the backend

Step 1: Create a React application using Vite.

```
npm create vite@latest user-frontend
```

```
D:\>cd Mern
D:\MERN> cd Prac9
D:\MERN\Prac9>npm create vite@latest user-frontend
> npx
> create-vite user-frontend
|
o  Select a framework:
|  React
|
o  Select a variant:
|  JavaScript
|
o  Which linter to use?
|  ESLint
|
o  Install with npm and start now?
|  Yes
|
o  Scaffolding project in D:\MERN\Prac9\user-frontend...
|
o  Installing dependencies with npm...
added 143 packages, and audited 144 packages in 10s
31 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
|
o  Starting dev server...
> user-frontend@0.0.0 dev
> vite
  VITE v8.3.3  ready in 536 ms
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
Terminate batch job (Y/N)? y
```

Step 2: Install the required packages.

```
D:\MERN\Prac9\user-frontend>npm install express mongoose cors
added 85 packages, and audited 229 packages in 3s
61 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

Step 3: Check package.json and start the React application.

```
D:\MERN\Prac9\user-frontend>npm init -y
Wrote to D:\MERN\Prac9\user-frontend\package.json:
{
  "name": "user-frontend",
```

```
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "express": "^5.2.1",
    "mongoose": "^9.11.0",
    "react": "^19.2.8",
    "react-dom": "^19.2.8"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "eslint": "^10.10.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.6",
    "globals": "^17.12.0",
    "vite": "^8.3.0"
  },
  "description": "This template provides a minimal setup to get React working in
Vite with HMR and some ESLint rules.",
  "main": "eslint.config.js",
  "keywords": [],
  "author": "",
  "license": "ISC"
}
D:\MERN\Prac9\user-frontend>npm run dev
```

```
npm run dev
```

```
D:\MERN\Prac9\user-frontend>npm run dev
> user-frontend@0.0.0 dev
> vite
1:18:49 PM [vite] (client) Re-optimizing dependencies because lockfile has
changed
  VITE v8.3.3  ready in 261 ms
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
1:19:31 PM [vite] (client) hmr update /src/App.jsx
1:19:31 PM [vite] (client) hmr update /src/App.jsx (x2)
1:19:31 PM [vite] (client) hmr update /src/App.jsx (x3)
Terminate batch job (Y/N)? y
```

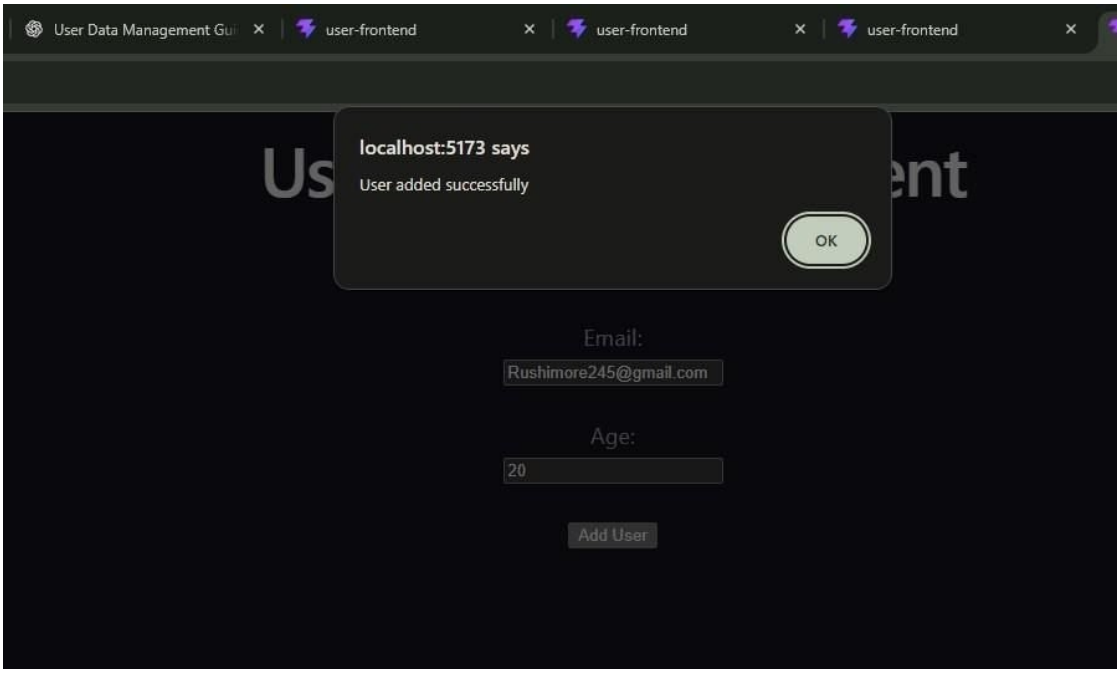
Step 4: Open src/App.jsx and paste the following code.

App.jsx

```
import React, { useState } from "react";
function App() {
  // Store form values
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [age, setAge] = useState("");
  // Send user data to Express backend
  const addUser = async (e) => {
    e.preventDefault();
    try {
      const response = await fetch("http://localhost:5000/users", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          name: name,
          email: email,
          age: age
        })
      });
      const data = await response.json();
      alert(data.message);
      // Clear the form
      setName("");
      setEmail("");
      setAge("");
    } catch (error) {
      console.log(error);
      alert("Error adding user");
    }
  };
  return (
    <div>
      <h1>User Data Management</h1>
      <form onSubmit={addUser}>
        {/* Name */}
        <label>Name:</label>
        <br />
        <input
          type="text"
          placeholder="Enter Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          required
        />
        <br />
        <br />
        {/* Email */}
        <label>Email:</label>
        <br />
        <input
```

```
          type="email"
          placeholder="Enter Email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
        <br />
        <br />
        {/* Age */}
        <label>Age:</label>
        <br />
        <input
          type="number"
          placeholder="Enter Age"
          value={age}
          onChange={(e) => setAge(e.target.value)}
          required
        />
        <br />
        <br />
        {/* Submit Button */}
        <button type="submit">
          Add User
        </button>
      </form>
    </div>
  );
}
export default App;
```

OUTPUT



User data stored in MongoDB (db.users.find()):

```
userdb> db.users.find();
[
  { _id: ObjectId('6ac5f4e416c28416bbecc8ac'), __v: 0 },
  {
    _id: ObjectId('6ac5f5b316c28416bbecc8ad'),
    name: 'Jatin',
    email: 'JAtin@gmail.com',
    age: 22,
    __v: 0
  },
  {
    _id: ObjectId('6ac5fa86b138fc07f0df2dfa'),
    name: 'jatin',
    email: 'shifashaikh501@gmail.com',
    age: 22,
    __v: 0
  },
  {
    _id: ObjectId('6ac5fb9ab138fc07f0df2dfb'),
    name: 'Rushikesh',
    email: 'Rushimore245@gmail.com',
    age: 20,
    __v: 0
  }
]
userdb>
```

9(c): Retrieve and display user data from MongoDB

Step 1: Check package.json of the frontend project.

```
D:\MERN\Prac9\user-frontend>npm init -y
Wrote to D:\MERN\Prac9\user-frontend\package.json:
{
  "name": "user-frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "express": "^5.2.1",
    "mongoose": "^9.11.0",
    "react": "^19.2.8",
    "react-dom": "^19.2.8"
  },
  "devDependencies": {
```

```
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "eslint": "^10.10.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.6",
    "globals": "^17.12.0",
    "vite": "^8.3.0"
  },
  "description": "This template provides a minimal setup to get React working in
Vite with HMR and some ESLint rules.",
  "main": "eslint.config.js",
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Step 2: Start the backend server.

```
node server.js
```

```
D:\MERN\Prac9\server>node server.js
Server running on port 5000
MongoDB Connected
```

Step 3: Start the React application.

```
npm run dev
```

```
D:\MERN\Prac9\user-frontend>npm run dev
> user-frontend@0.0.0 dev
> vite
1:18:49 PM [vite] (client) Re-optimizing dependencies because lockfile has
changed
  VITE v8.3.3  ready in 261 ms
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
1:19:31 PM [vite] (client) hmr update /src/App.jsx
1:19:31 PM [vite] (client) hmr update /src/App.jsx (x2)
1:19:31 PM [vite] (client) hmr update /src/App.jsx (x3)
Terminate batch job (Y/N)? y
```

Step 4: Edit App.jsx and add the following code.

App.jsx

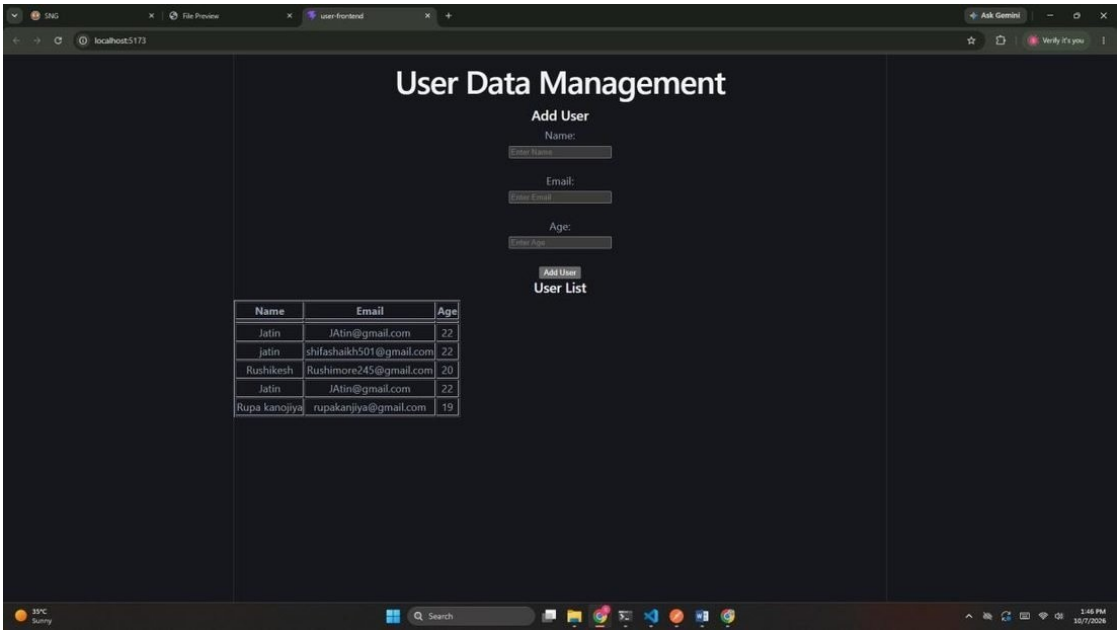
```
import React, { useState, useEffect } from "react";
function App() {
  // Form values
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [age, setAge] = useState("");
  // Store users retrieved from MongoDB
  const [users, setUsers] = useState([]);
```

```
// Get users from backend
const getUsers = async () => {
  try {
    const response = await fetch(
      "http://localhost:5000/users"
    );
    const data = await response.json();
    setUsers(data);
  } catch (error) {
    console.log(error);
    alert("Error getting users");
  }
};
// Run getUsers when page loads
useEffect(() => {
  getUsers();
}, []);
// Add user
const addUser = async (e) => {
  e.preventDefault();
  try {
    const response = await fetch(
      "http://localhost:5000/users",
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          name: name,
          email: email,
          age: age
        })
      }
    );
    const data = await response.json();
    alert(data.message);
    // Clear form
    setName("");
    setEmail("");
    setAge("");
    // Refresh user list
    getUsers();
  } catch (error) {
    console.log(error);
    alert("Error adding user");
  }
};
return (
  <div>
    <h1>User Data Management</h1>
    {/* User Form */}
    <h2>Add User</h2>
    <form onSubmit={addUser}>
```

```
        <label>Name:</label>
        <br />
        <input
          type="text"
          placeholder="Enter Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          required
        />
        <br />
        <br />
        <label>Email:</label>
        <br />
        <input
          type="email"
          placeholder="Enter Email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
        <br />
        <br />
        <label>Age:</label>
        <br />
        <input
          type="number"
          placeholder="Enter Age"
          value={age}
          onChange={(e) => setAge(e.target.value)}
          required
        />
        <br />
        <br />
        <button type="submit">
          Add User
        </button>
      </form>
      {/* Display Users */}
      <h2>User List</h2>
      <table border="1">
        <thead>
          <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Age</th>
          </tr>
        </thead>
        <tbody>
          {users.map((user) => (
            <tr key={user._id}>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.age}</td>
            </tr>
```

```
          ))}
        </tbody>
      </table>
    </div>
  );
}
export default App;
```

OUTPUT



9(d): Implement an update operation for existing records

Step 1: Update your server.js.

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/userManagement")
  .then(() => console.log("MongoDB Connected"))
  .catch((error) => console.log("MongoDB Error:", error));
// User Schema
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
```

```
    required: true
  },
  age: {
    type: Number,
    required: true
  }
});
// User Model
const User = mongoose.model("User", userSchema);
// API to add a user
app.post("/users", async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).json({
      message: "User added successfully",
      user: user
    });
  } catch (error) {
    console.error("POST error:", error);
    res.status(500).json({
      message: "Error adding user",
      error: error.message
    });
  }
});
// API to retrieve all users
app.get("/users", async (req, res) => {
  try {
    const allUsers = await User.find();
    res.status(200).json(allUsers);
  } catch (error) {
    console.error("GET error:", error);
    res.status(500).json({
      message: "Error retrieving users",
      error: error.message
    });
  }
});
// API to update an existing user
app.put("/users/:id", async (req, res) => {
  try {
    const updatedUser = await User.findByIdAndUpdate(
      req.params.id,
      {
        name: req.body.name,
        email: req.body.email,
        age: req.body.age
      },
      {
        new: true,
        runValidators: true
      }
    );
```

```
    if (!updatedUser) {
      return res.status(404).json({
        message: "User not found"
      });
    }
    res.status(200).json({
      message: "User updated successfully",
      user: updatedUser
    });
  } catch (error) {
    console.error("PUT error:", error);
    res.status(400).json({
      message: "Error updating user",
      error: error.message
    });
  }
});
// Start Server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Restart the server:

PowerShell

```
PS D:\Sinuuu\prac9\server> node .\server.js
Server running on port 5000
MongoDB Connected
```

Step 2: Update App.jsx.

Now open: Practical9/user-frontend/src/App.jsx

We need to add edit functionality while keeping your existing form, GET API, and POST API working. Replace the entire App.jsx file with this complete version:

App.jsx

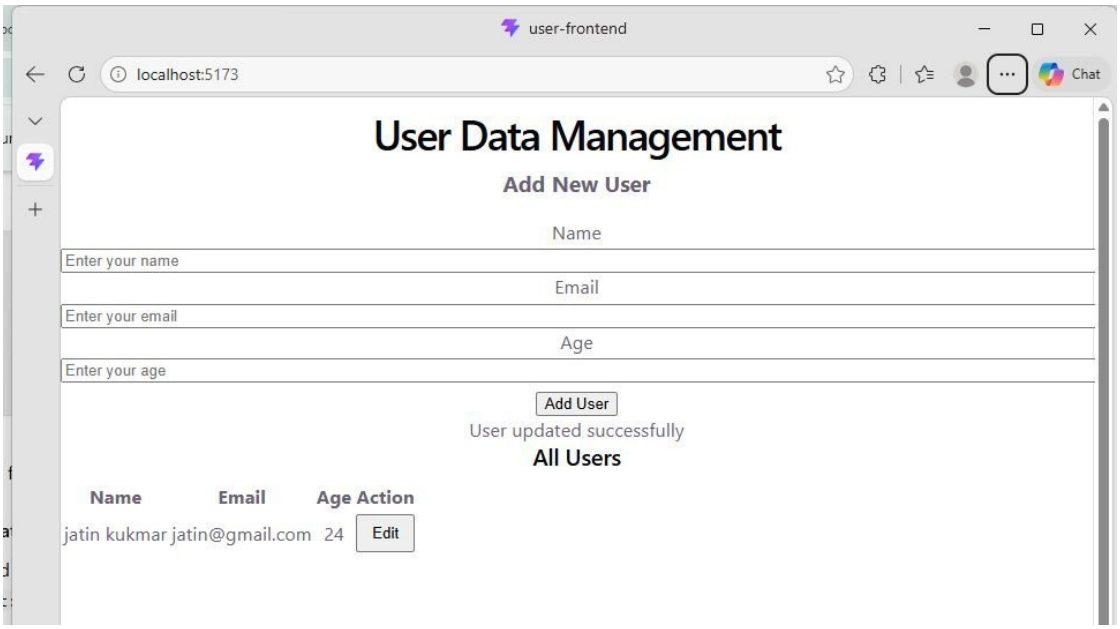
```
import { useState, useEffect } from "react";
import "./App.css";
function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [age, setAge] = useState("");
  const [users, setUsers] = useState([]);
  const [message, setMessage] = useState("");
  const [editId, setEditId] = useState(null);
  // Retrieve all users
  const fetchUsers = async () => {
    try {
      const response = await fetch("http://localhost:5000/users");
      if (!response.ok) {
        throw new Error("Failed to retrieve users");
      }
```

```
    const data = await response.json();
    setUsers(data);
  } catch (error) {
    console.error(error);
    setMessage("Unable to load users.");
  }
};
useEffect(() => {
  fetchUsers();
}, []);
// Add or update user
const handleSubmit = async (e) => {
  e.preventDefault();
  setMessage("");
  const userData = {
    name,
    email,
    age: Number(age),
  };
  try {
    const url = editId
      ? `http://localhost:5000/users/${editId}`
      : "http://localhost:5000/users";
    const response = await fetch(url, {
      method: editId ? "PUT" : "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(userData),
    });
    const data = await response.json();
    if (!response.ok) {
      throw new Error(data.message || "Operation failed");
    }
    setMessage(data.message);
    clearForm();
    await fetchUsers();
  } catch (error) {
    console.error(error);
    setMessage(error.message || "Cannot connect to backend.");
  }
};
// Load selected user into the form
const handleEdit = (user) => {
  setEditId(user._id);
  setName(user.name);
  setEmail(user.email);
  setAge(String(user.age));
  setMessage("Edit the details and click Update User.");
};
// Clear form
const clearForm = () => {
  setName("");
  setEmail("");
```

```
    setAge("");
    setEditId(null);
  };
  return (
    <div className="container">
      <h1>User Data Management</h1>
      <h3>{editId ? "Update User" : "Add New User"}</h3>
      <form onSubmit={handleSubmit}>
        <label>Name</label>
        <input
          type="text"
          placeholder="Enter your name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          required
        />
        <label>Email</label>
        <input
          type="email"
          placeholder="Enter your email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
        <label>Age</label>
        <input
          type="number"
          placeholder="Enter your age"
          value={age}
          onChange={(e) => setAge(e.target.value)}
          min="1"
          required
        />
        <button type="submit">
          {editId ? "Update User" : "Add User"}
        </button>
        {editId && (
          <button
            type="button"
            onClick={clearForm}
            className="cancel-btn"
          >
            Cancel Edit
          </button>
        )}
      </form>
      {message && <p>{message}</p>}
      <h2>All Users</h2>
      {users.length === 0 ? (
        <p>No users found.</p>
      ) : (
        <div className="table-container">
          <table>
            <thead>
```

```
              <tr>
                <th>Name</th>
                <th>Email</th>
                <th>Age</th>
                <th>Action</th>
              </tr>
            </thead>
            <tbody>
              {users.map((user) => (
                <tr key={user._id}>
                  <td>{user.name}</td>
                  <td>{user.email}</td>
                  <td>{user.age}</td>
                  <td>
                    <button
                      type="button"
                      onClick={() => handleEdit(user)}
                    >
                      Edit
                    </button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </div>
  );
}
export default App;
```

OUTPUT



9(e): Implement the delete operation for selected records

Step 1: Create the DELETE API.

Open Practical9/backend/server.js. Find the // Start Server comment. Add this code just above it, after your PUT API.

server.js

```
// API to delete a user
app.delete("/users/:id", async (req, res) => {
  try {
    const deletedUser = await User.findByIdAndDelete(
      req.params.id
    );
    if (!deletedUser) {
      return res.status(404).json({
        message: "User not found"
      });
    }
    res.status(200).json({
      message: "User deleted successfully"
    });
  } catch (error) {
    console.error("DELETE error:", error);
    res.status(400).json({
      message: "Error deleting user",
      error: error.message
    });
  }
});
```

Step 2: Add the delete function in React.

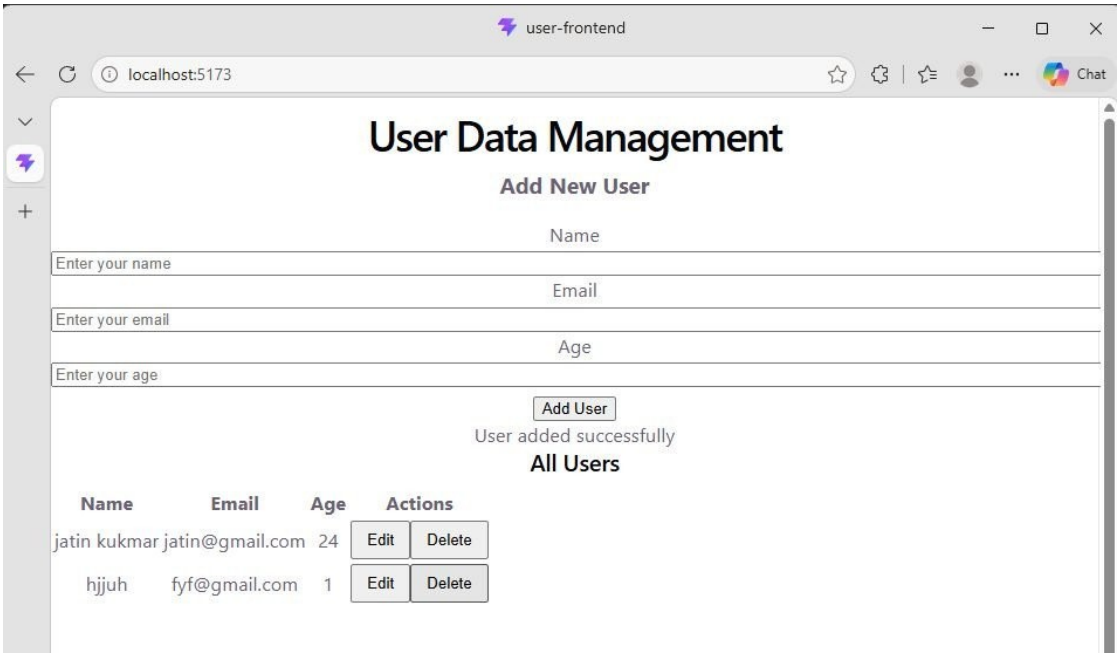
Open Practical9/user-frontend/src/App.jsx. Find the handleEdit function. Immediately below it, add this function:

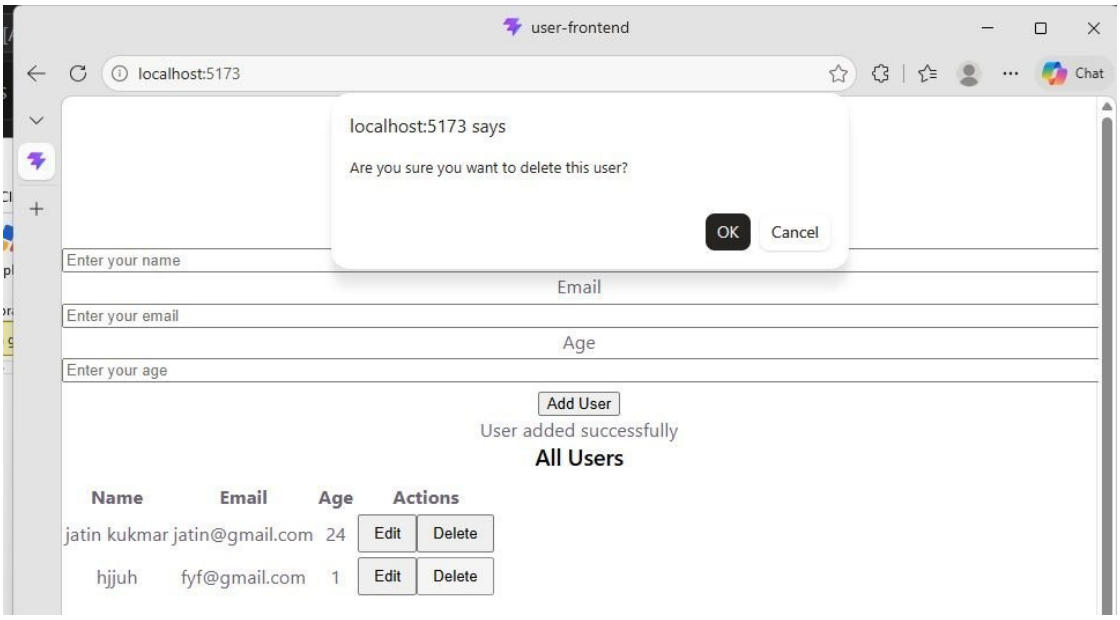
App.jsx

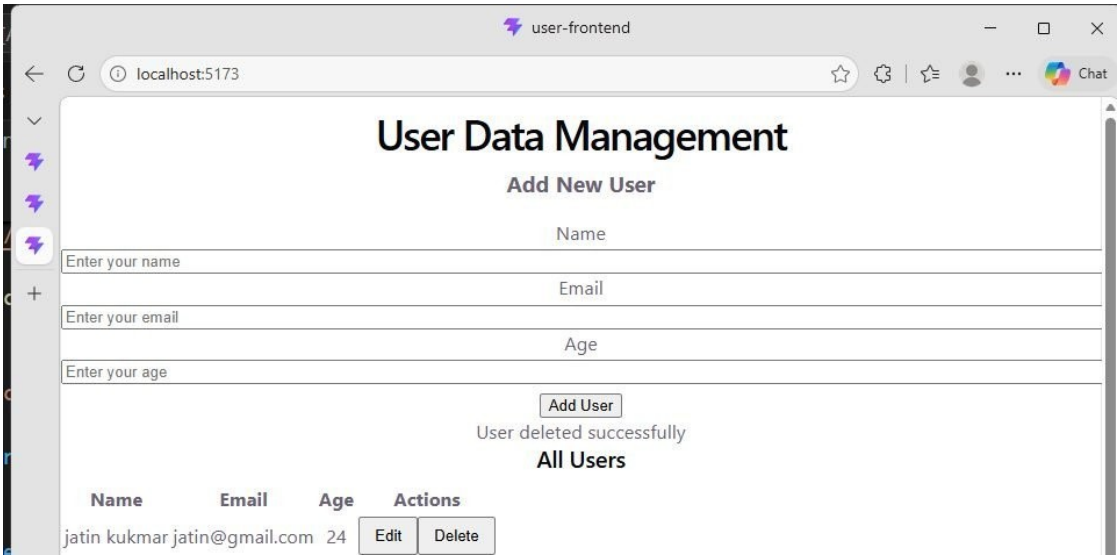
```
// Delete a selected user
const handleDelete = async (id) => {
  const confirmed = window.confirm(
    "Are you sure you want to delete this user?"
  );
  if (!confirmed) return;
  try {
    const response = await fetch(
      `http://localhost:5000/users/${id}`,
      {
        method: "DELETE",
      }
    );
    const data = await response.json();
    if (!response.ok) {
      throw new Error(data.message || "Failed to delete user");
```

```
    }
    setMessage(data.message);
    // Clear the form if the deleted user was being edited
    if (editId === id) {
      clearForm();
    }
    // Refresh the table
    await fetchUsers();
  } catch (error) {
    console.error(error);
    setMessage(error.message || "Cannot connect to backend.");
  }
};
```

OUTPUT




user-frontend
localhost:5173
Chat
localhost:5173 says
Are you sure you want to delete this user?
OK
Cancel
Enter your name
Email
Enter your email
Age
Enter your age
Add User
User added successfully
All Users
Name Email Age Actions
jatin kukmar jatin@gmail.com 24 Edit Delete
hjjuh fyf@gmail.com 1 Edit Delete


user-frontend
localhost:5173
Chat
User Data Management
Add New User
Name
Enter your name
Email
Enter your email
Age
Enter your age
Add User
User deleted successfully
All Users
Name Email Age Actions
jatin kukmar jatin@gmail.com 24 Edit Delete